

*A project report submitted to ICT Academy of Kerala
in partial fulfillment of the requirements
for the certification of*

**CERTIFIED SPECIALIST
IN
DATA SCIENCE & ANALYTICS**

submitted by

**Team members
Chithra KS
Sona P
Sureshababu Thekketil**



**ICT ACADEMY OF KERALA
THIRUVANANTHAPURAM, KERALA, INDIA
JULY 2024**

List of Figures

Figure No.	Pg.No
1. Count plot of target.....	11
2. DOB year VS PotentialFraud.....	12
3. DOD year vs PotentialFraud.....	13
4. countplot of Gender vs potentialfraud.....	13
5. countplot of Race vs potentialfraud.....	14
6. State VS potential fraud bar plot.....	14
7. Barplot county vs potentialfraud.....	15
8. crosstab plot renalDiseaseIndicator vs potentialfraud.....	15
9. Barplot potential fraud VS NoOfMonths_PartACov.....	16
10. Barplot between potential fraud and NoOfMonths_PartBCov.....	16
11. countplot of claimstartdate.....	17
12. Admissiondate barplot.....	18
13. Barplot DischargeDt va target.....	18
14. countplot of DischargeDt.....	19
15. Heatmap of provider Vs potentialfraud.....	19
16. countplot of Attendingphysician.....	20
17. Countplot of operatingphysician.....	20
18. Countplot of otherphysician.....	21
19. Barplot of InscClaimAmtReimbursed vs Target.....	21
20. histogram of DeductibleamtPaid	22
21.. Barplot of IPAnnualReimbursementAmt vs target.....	22
22. barplot of IPAnnualDeductibleAmt vs target.....	23
23. Barplot of OPAnnualReimbursementAmt vs target.....	23
24. Barplot of OPAnnualDeductibleAmt vs target.....	24
25. boxplot of op and IP deductible and reimbursed amount	24
26. Histogram of op and ip reimbursed and deductible amount.....	25
27. heatmap of claimdiagnosiscode vs potential fraud.....	25
28. Histogram of claim procedure codes.....	26
29. Barplot of ChronicCond_Alzheimer vs target.....	27
30. Barplot ChronicCond_Heartfailure vs potentialfraud.....	28
31. Barplot of ChronicCond_KidneyDisease vs Target.....	28
32. Barplot of ChronicCond_Cancer vs Target.....	29
33. Barplot of ChronicCond_ObstrPulmonary vs Target.....	29
34. Barplot of ChronicCond_Depression vs target.....	30
35. Histogram chronicconditions.....	30
36. countplot of chronicdiabetes vs potentialfraud.....	31
37. count plot of chroniccond_ischemicHeart vs potential fraud.....	31

38. countplot of chroniccond_osteoporosis vs potentialfraud.....	32
39. barplot of ChronicCond_rheumatoidarthritis vs Target.....	32
40. Barplot ChronicCond_stroke vs target.....	33
41. Heatmap of correlation matrix.....	33
42. Boxplot of Clmprocedurecode 1 and Clmprocedurecode 2.....	35
43: Box plot of TotalClaimAm.....	36
44: Boxplot of optotalamt.....	37
45: Boxplot of TotalAnnualIPamount.....	38
46: Box plot of physician appearing frequency.....	39
47. Correlation matrix after feature engineering.....	40
48: Precision-Recall curve of ADA boost classifier for normaldata.....	44
49: Precision-Recall curve of Light GBM classifier for normal data.....	45
50: Precision-Recall curve of ADA boostclassifier for oversampled data.....	47
51: Precision-Recall curve of Light GBM classifier for oversampled data.....	47
52. Precision -Recall curve XGboost for Original Data without Hyperparameter Tuning.....	51
53. Precision -Recall curve XGboost for Data without Hyperparameter Tuning.....	51
54. Precision-Recall curve for Over Sampled Data without Hyperparameter Tunin .	52
55. Precision-Recall curve for Over Sampled Data without Hyperparameter Tuning.....	53
56. Performance metricsst stacked barplot for various models.....	53
57. Feature Importance derived from Test data.....	54
58. Histogram of Predicted labels in web page.....	55
59. Website Homepage.....	55
60. Provider Fraud detection Result display in web.....	56

List of Tables

	Pg.No.
1. Target label	11
2. crosstab of claimdiagnosisgroupcode vs target.....	27
3. ANOVA Test Result-Columns with P value > 0.05.....	40
4.Target label proportion after oversampling.....	42
5. Description of oversampled data after scaling.....	43
6. Performance metrics of ADA boost classifier (data without over-sampling).....	44
7. Performance metrics of Light GBM classifier (data without over-sampling).....	45
8.Performance metrics of ADA boost classifier (data with over-sampling).....	46
9. Performance metrics of Light GBM classifier (data with over-sampling).....	47

List of Abbreviations

- **SMOTE** – Synthetic Minority Over-sampling Technique
- **KNN** – K-Nearest Neighbors
- **Random Forest (RF)** – Random Forest Classifier
- **Decision Tree (DT)** – Decision Tree Classifier
- **Naive Bayes (NB)** – Naive Bayes Classifier
- **XGBoost (XGB)** – Extreme Gradient Boosting
- **LightGBM (LGBM)** – Light Gradient Boosting Machine
- **AdaBoost** – Adaptive Boost

Table of Contents

	Pg. No.
● ABSTRACT.....	7
● PROBLEM DEFINITION.....	8
● INTRODUCTION.....	9
● METHODOLOGY.....	10
● FEATURE ENGINEERING.....	34
● MODELING.....	43
● EVALUATION.....	50
● RESULT.....	54
● DISCUSSION.....	57
● CONCLUSION.....	59
● REFERENCES.....	60

Abstract

Health care fraud remains a significant challenge in the medical industry, Healthcare provider fraud is one of the biggest problems facing Medicare. The total Medicare expenditure increased exponentially leading to substantial financial loss and compromised patient care. This project focuses on developing a robust system for health care fraud detection through machine learning (ML) modelling and creating a web application to facilitate user interaction and decision-making.

Problem Definition

Objective and Problem Statement

The objective is to develop a machine learning (ML) model to predict fraudulent transactions among healthcare providers and to create a web application using Python Flask to facilitate the same.

Introduction

Fraud detection is a critical component in the insurance industry, aiming to identify and mitigate fraudulent activities that lead to significant financial losses. Insurance fraud occurs when individuals or entities provide false information or manipulate claims for personal or financial gain, thereby increasing the overall cost of insurance for all policyholders.

This project focuses on fraud detection analysis in the insurance domain, leveraging advanced data analytics, machine learning, and statistical methods to identify suspicious activities or patterns in claims data. The primary goal is to develop a robust system that can accurately detect potential fraud, improve operational efficiency, and reduce financial losses for insurance companies. Finally, we are developing a web application using PYTHON FLASK and hosting it as a website using PythonAnywhere.

Methodology

- **Data Acquisition**: The data for this project was collected from ‘Kaggle’ and comprises multiple datasets.
 1. Inpatient Data: This data provides insights into the claims filed for those patients who are admitted in the hospitals. It also provides additional details like their admission and discharge dates and admitted diagnosis code. **(40474*30)**
 2. Outpatient Data: This data provides details about the claims filed for those patients who visit hospitals and are not admitted to it. **(517737*27)**
 3. Beneficiary Details Data: This data contains beneficiary KYC details like health conditions, region they belong to etc. **(138556* 25)**
 4. Fraudulent transaction dataset: This dataset provides information about the fraudulent transaction by the health care provider. **(5410*2)**
- Inpatient and outpatient datasets contain similar columns, except for two, so we ‘concatenated’ these two datasets. Next, we ‘merged’ the beneficiary dataset with this combined dataset on ‘Beneficiary ID’ column. The resulting dataset is then ‘merged’ with the fraudulent transaction dataset on ‘Provider’ column. An Outer merger was done to preserve all observations from both the datasets. The final dataset we got has 558211 observations 55 features **(558211 *55)**.
- **Exploratory Data Analysis (EDA)**:

Dataset overview

Our dataset consists of **558,211 observations** and **55 features**. We have performed **univariate analysis** on each feature, as well as **bivariate analysis** with the target column, to assess their influence on the target variable. There are 29 numerical columns, some

of which are discrete numerical columns, while others are continuous. The remaining 26 columns are categorical, including 4 date columns that were converted into datetime variables.

The target column, '**PotentialFraud**', is of **object type** and contains **two unique values**, namely '**YES**' and '**NO**'. The proportion of these values is as follows,

proportion	
PotentialFraud	
No	0.618789
Yes	0.381211

Table1: Target label proportion

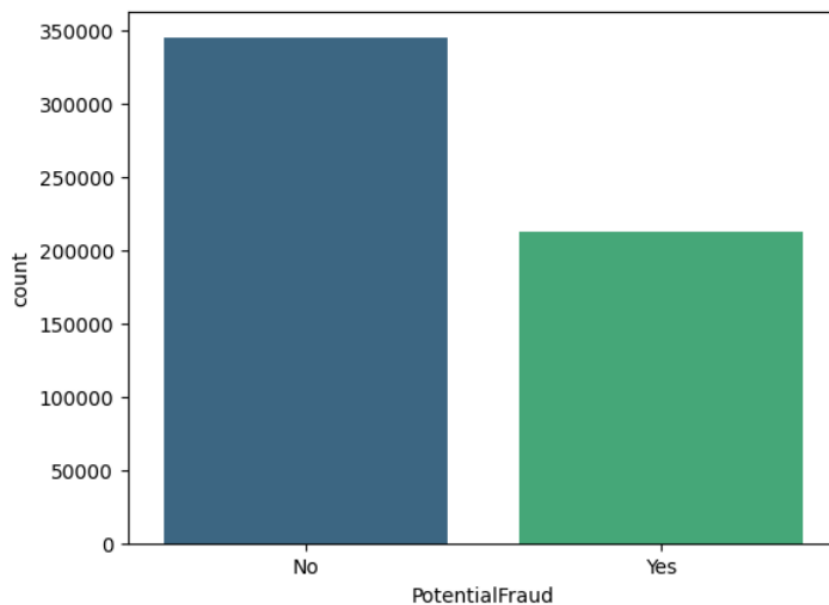


Fig:1. Countplot of target.

Our target column is slightly biased toward '**NO**'. To ensure our model makes accurate predictions without favouring '**NO**', we need to address this imbalance during the preprocessing steps.

Univariate and Bivariate analysis findings:

Patient and Demographic Information:

1. BeneID

Unique identifier for the beneficiary (patient). Helps track patient-level data. 55821 unique 'BeneIDs' are there. After doing chi-square test we find out that this feature has statistically significant association with the target.

2. **DOB:** Date of birth of the patient. Useful for deriving age, which can be an important feature. This is an 'Object' type column with 55821 observations, i.e. with no missing values. After performing the chi-square test, we find out that DOB has statistically significant relation with the target.

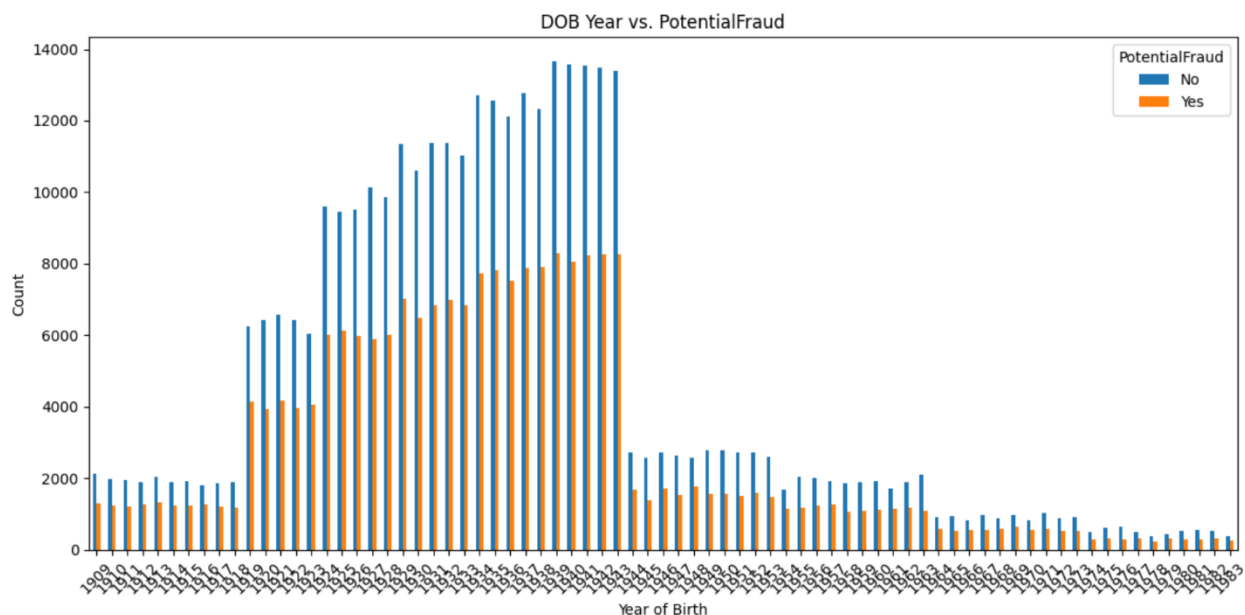


Fig.2: DOB year VS PotentialFraud.

3. **DOD:** Date of death of the patient, if applicable. Useful for identifying inconsistencies in claims (e.g., claims filed after death). It is an 'Object' type column with only 4131 non-null values and all other cells have null values, we have to deal with it in preprocessing. There is a statistically significant association with targets.

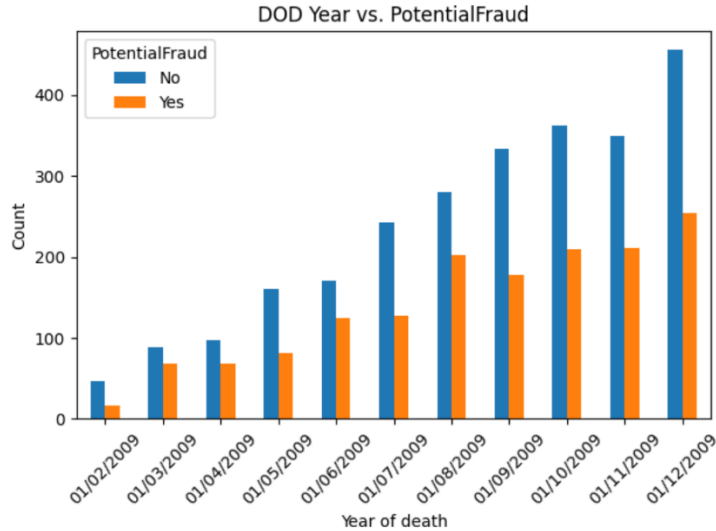


Fig 3: DOD year vs Potentialfraud.

4. **Gender:** Patient's gender (e.g., Male, Female). A demographic factor that may influence health conditions or fraud patterns. In our dataset it is represented as 1 and 2, and it's a discrete numerical column.no null values present ,558211 observations are there.

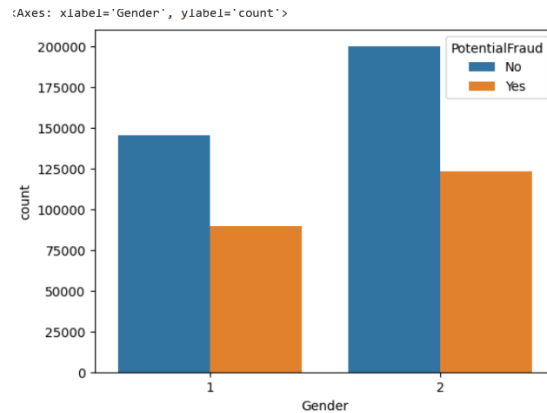


Fig 4: countplot of Gender vs potentialfraud

5. **Race:** Race or ethnicity of the patient. Could provide insights into claim trends or disparities. This is also an integer column with no null values and has 4 values an array ([1, 2, 5, 3]) which indicates different races. The relationship of race with our target is plotted below.

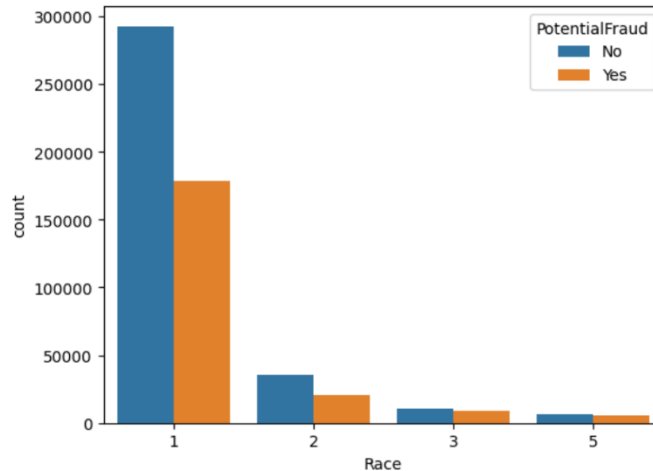


Fig 5:countplot of Race vs potentialfraud

6. **State:** State where the patient resides. Helps identify geographic fraud trends. Its data type is int64, discrete numerical. Distribution is skewed. It has missed values and no outliers. It is relating to potential fraud as follows:

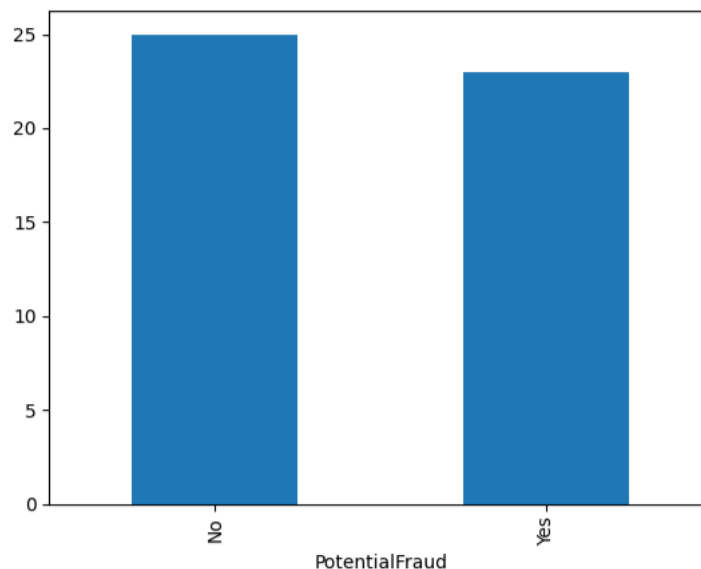


Fig:6. State VS potential fraud bar plot.

7. **County:** County of residence for the patient. Adds granularity to geographic Data. Its data type is int64, discrete numerical. Distribution is skewed. It has missing values and no outliers. It relates to potential fraud as follows:

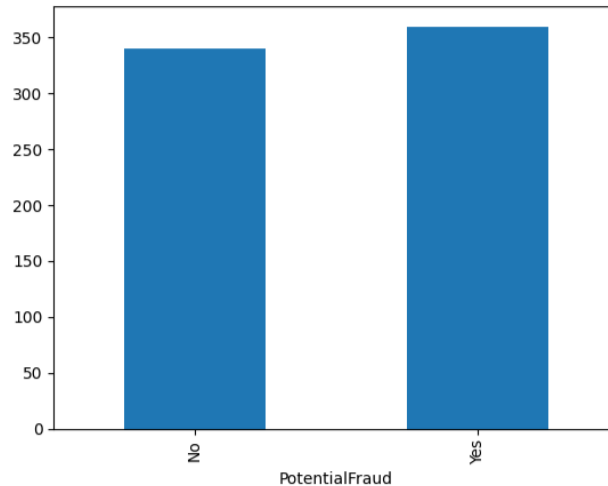


Fig:7. Barplot county vs potentialfraud

8. **RenalDiseaseIndicator:** Indicates if the patient has end-stage renal disease. Relevant for identifying patterns in medical needs and claims. This contains 2 unique value y(yes) and 0 (no).it is a categorical column with no null values, and there is a statistically significant association between RenalDiseaseIndicator and PotentialFraud.

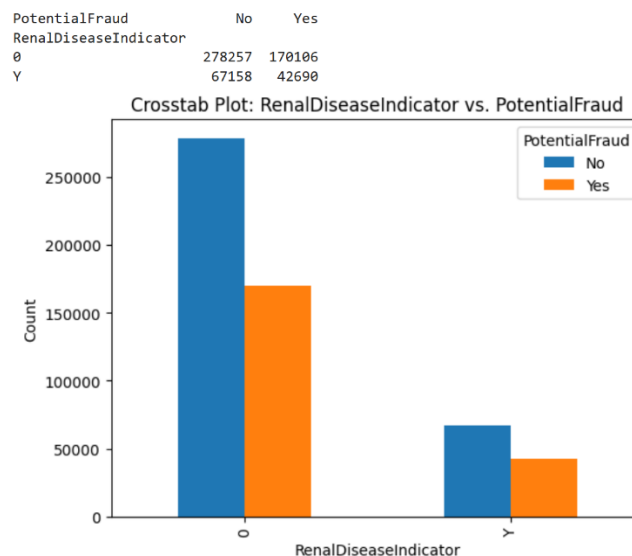


Fig:8.crosstab plot renaliseaseindicator vs potentialfraud.

9. **NoOfMonths_PartACov:** Number of months the patient was covered under Medicare Part A (hospital insurance). Its data type is int64, discrete numerical. Distribution is skewed.it has missing values and has outliers.it relates to potential fraud as follows:

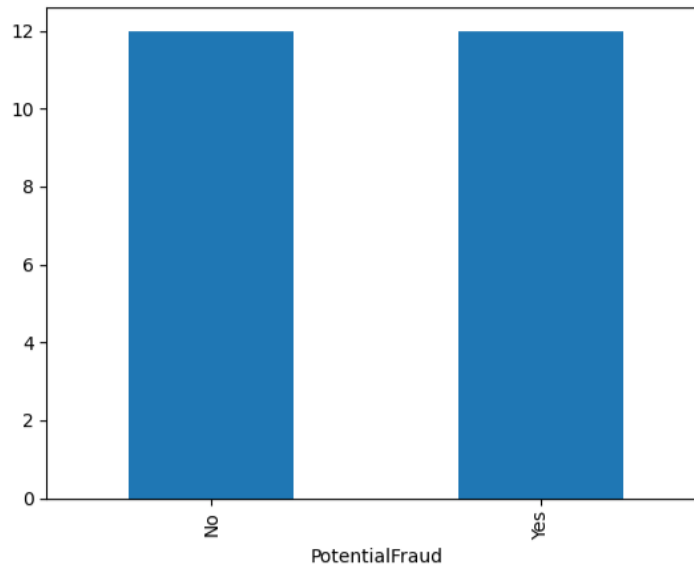


Fig:9.barplot potential fraud VS **NoOfMonths_PartACov**

10.NoOfMonths_PartBCov: Number of months the patient was covered under Medicare Part B (medical insurance). Its data type is int64, discrete numerical. Distribution is skewed.it has missing values and has outliers.it relates to potential fraud as follows:

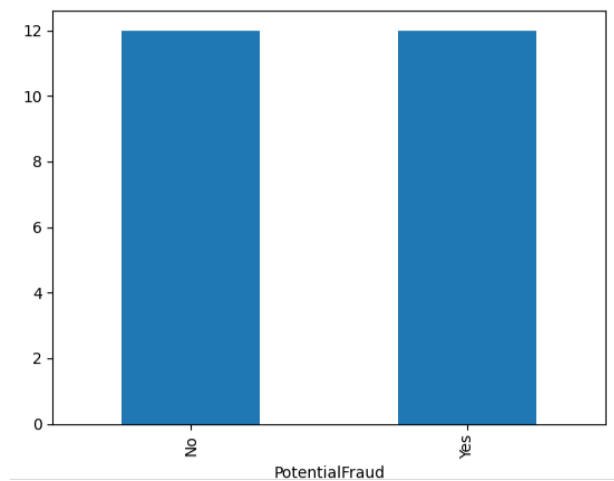


Fig:10..barplot between potential fraud and **NoOfMonths_PartBCov**

Claim Information

11.ClaimID

Unique identifier for a claim. Helps in tracking and linking data. There are 558211 unique ClaimID's there. After doing the chi-square test, we find out that there is no statistically significant association between ClaimID and PotentialFraud. Hence, we can drop this column during feature reduction step.

12.ClaimStartDt

Start date of the claim. Can be used to calculate claim duration. This column is a categorical column, we converted it into datetime datatype and carried out chi-square test and find out that there is a statistically significant association between ClaimStartDt and PotentialFraud. There is no null value, 558211 observations are there. From the countplot it observed that most of the claims are started on 2009 and some on 2008. This is a 2009-2008 based dataset.

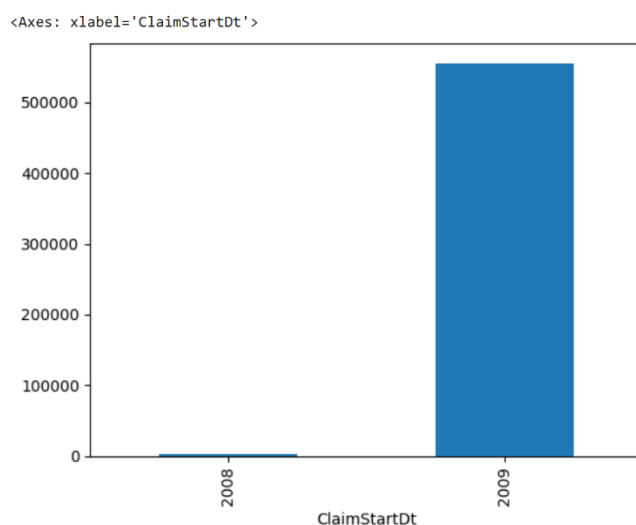


Fig 11: countplot of claimstartdate.

13.ClaimEndDt

End date of the claim. Combined with ClaimStartDt to check claim periods. Object column later converted into datetime datatype. There are no null values. After chi-square test, we find out that There is no statistically significant association between ClaimEndDt and PotentialFraud. Most of the claims are ending in 2009.

14. AdmissionDt

Date the patient was admitted to the hospital. Useful for verifying inpatient claims. Its data type is Object. We converted its data type to datetime. It has missing values and there is statistical significance with potential fraud.

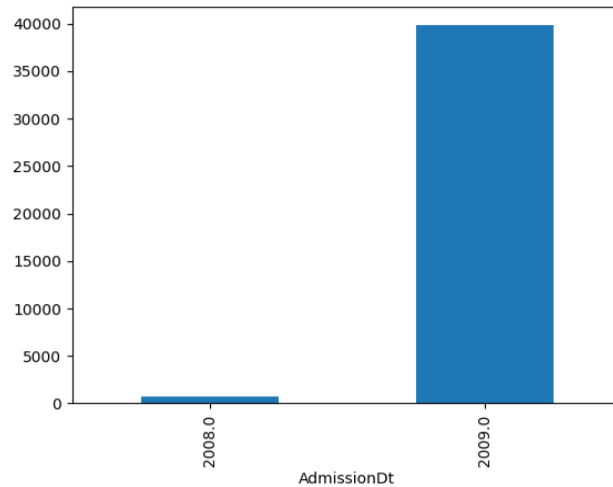


Fig.12. Admissiondate barplot

15. DischargeDt

Date the patient was discharged from the hospital. Can be cross-checked with admission and claim dates. There is missing values and no statistically significant association between Discharge date and Potential Fraud.

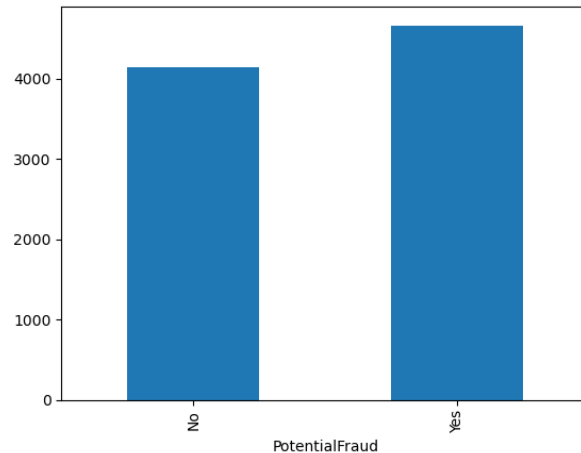


Fig 13: Barplot DischargeDt va target

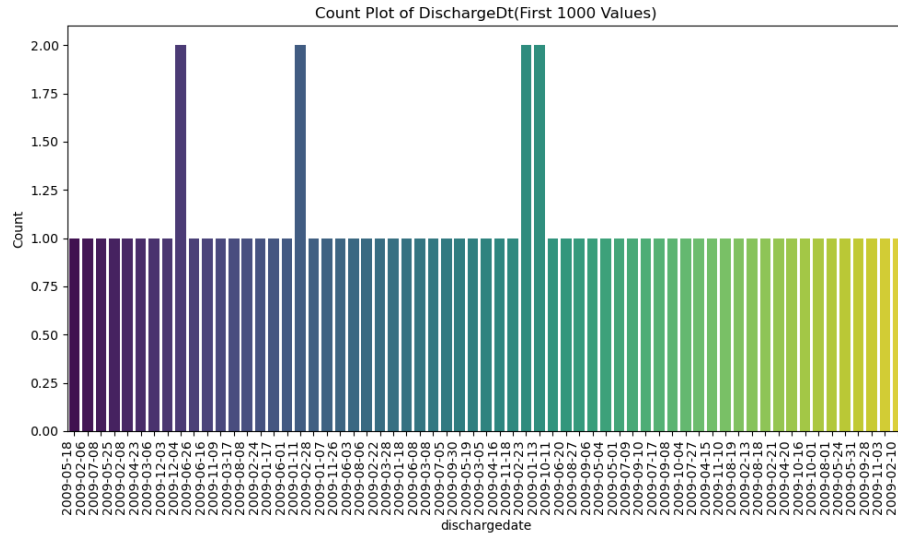


Fig.14:countplot of **DischargeDt**

16.Provider

Unique identifier for the healthcare provider. Helps in tracking provider-specific trends or fraudulent behavior. This feature is very important in detection of provider fraud .558211 unique providers are there, it is a categorical column. By doing chi-square test we find out there is a statistically significant association between Provider and PotentialFraud.

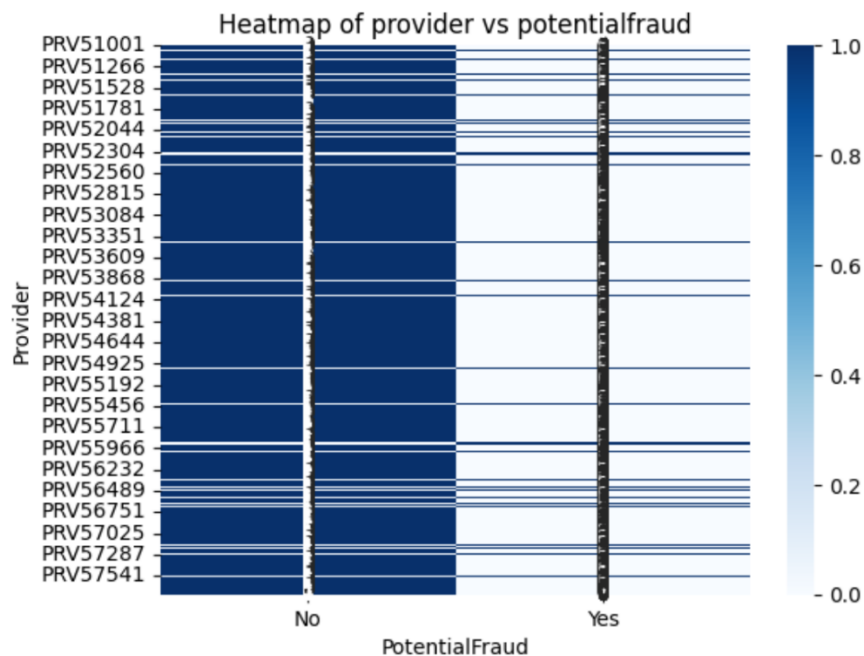


Fig: 15.Heatmap of provider Vs potentialfraud.

The above heatmap shows that among the 558211 providers most of them fall under the 'NO' category of our target which is marked as high intensity area in the heatmap.

17. **AttendingPhysician**

Identifier for the primary physician responsible for the patient's care. Its data type is Object. It has missing values and there is statistical significance with potential fraud.

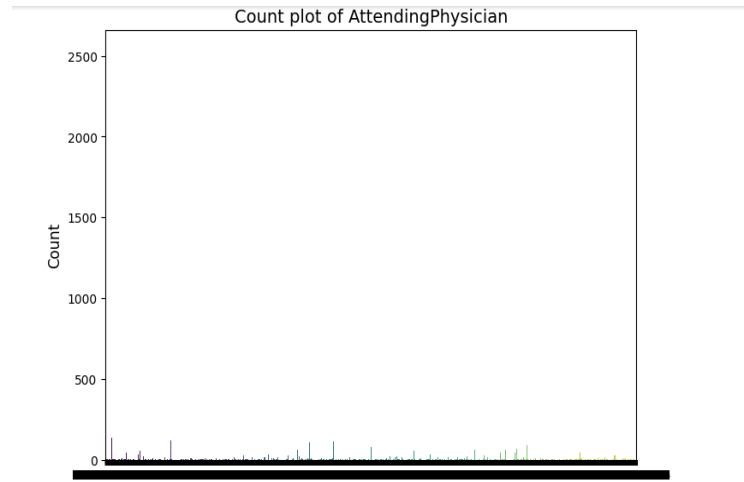


Fig16:..countplot of Attendingphysician

18. **OperatingPhysician**

Identifier for the physician who performed any surgeries or operations. Its data type is Object. It has missing values and there is statistical significance with potential fraud.

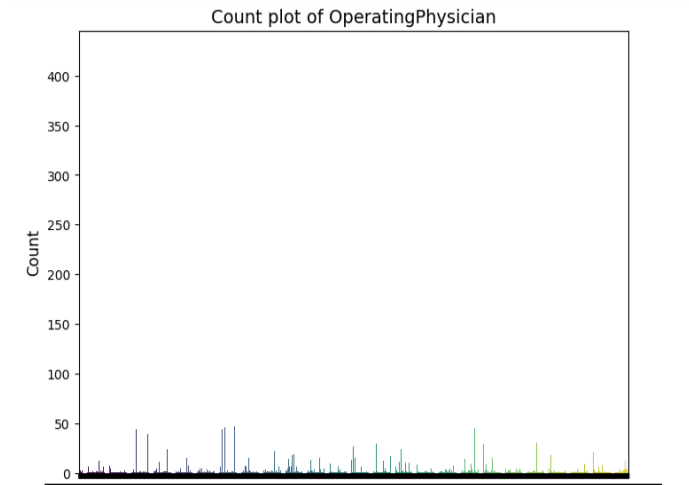


Fig:17:Countplot of operatingphysician

19. OtherPhysician

Identifier for other physicians involved in the patient's care. Its data type is Object. It has missing values and there is statistical significance with potential fraud.

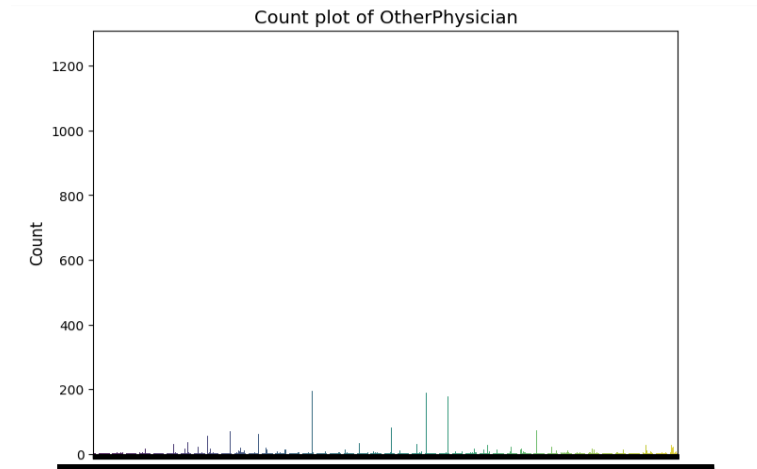


Fig:18:Countplot of otherphysician

Financial Information

20. InscClaimAmtReimbursed

Amount reimbursed by the insurer for the claim. its data type is int64, discrete numerical. It has no missing value but has outliers and it is a skewed distribution. High amount may signal potential Fraud.

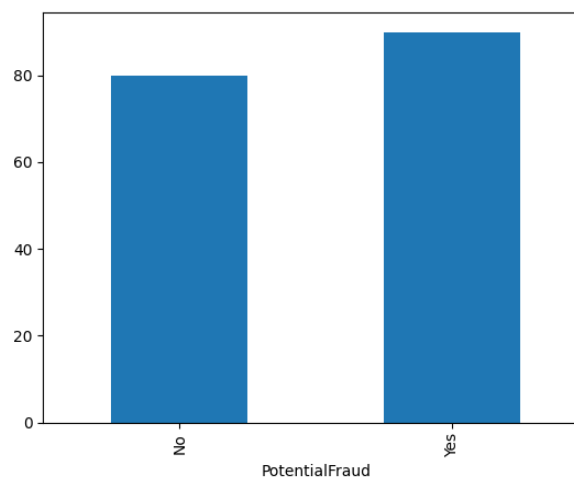


Fig:19.Barplot of InscClaimAmtReimbursed vs Target

20. DeductibleAmtPaid

Deductible amount paid by the patient. Unusually high or low values might be worth investigating. The box plot is empty due to missing values and below is the histogram presentation:

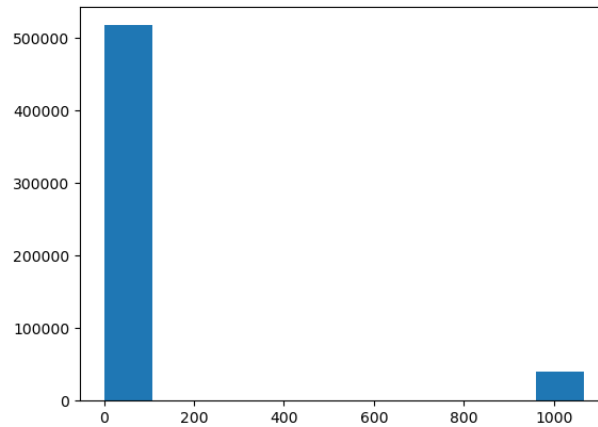


Fig:20.histogram of Deductibleamtpaid

21. IPAnnualReimbursementAmt

Annual amount reimbursed for inpatient services. Its data type is int64, discrete numerical. distribution is skewed. It has missing values and has outliers. It relates to potential fraud as follows:

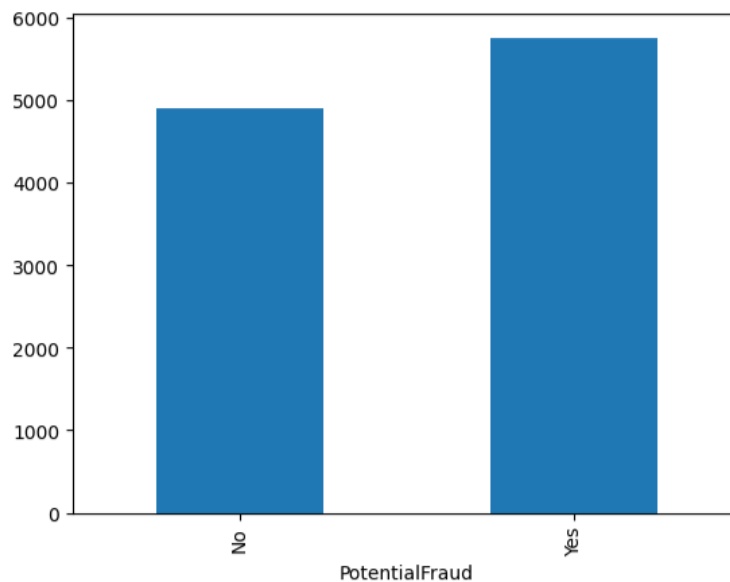


Fig:21.Barplot of IPAnnualReimbursementAmt vs target

22. IPAnnualDeductibleAmt: This is the total deductible amount applicable for inpatient care for a particular year. This amount is to be paid by the patient out of their own pocket.

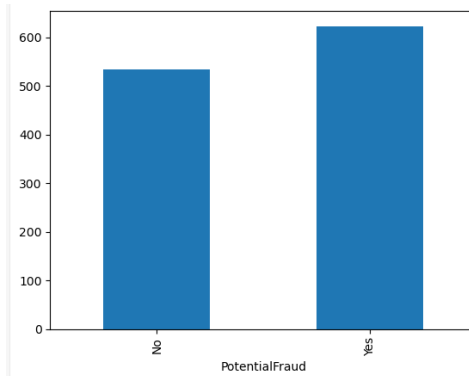


Fig: 22.barplot of IPAnnualDeductibleAmt vs target

23.OPAnnualReimbursementAmt

Annual amount reimbursed for outpatient services. It represents the total payment made to a hospital or healthcare provider by insurance company for outpatient services rendered during a year.

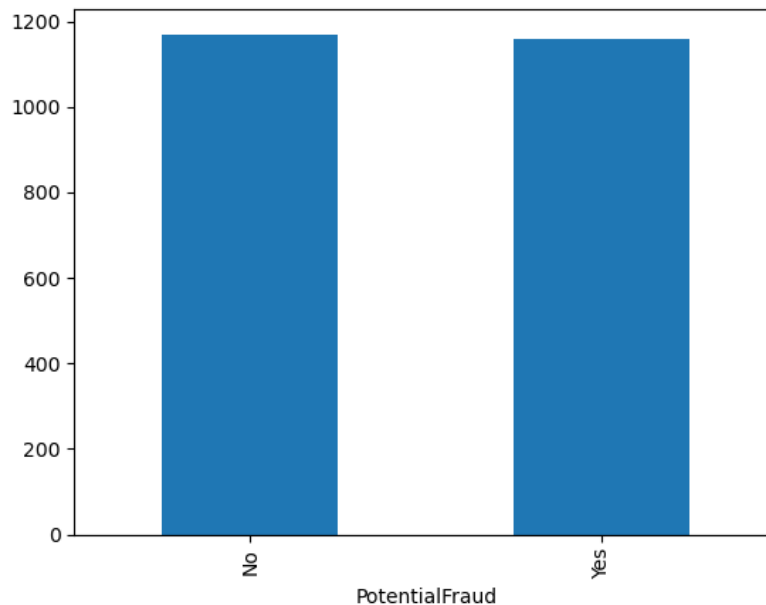


Fig23:Barplot of OPAnnualReimbursementAmt vs target

24.OPAnnualDeductibleAmt

Annual deductible paid for outpatient services. This is the total deductible amount applicable for outpatient care for a particular year. This amount is to be paid by the patient out of their own pocket.

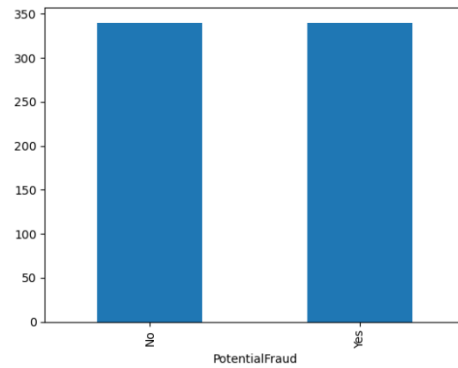


Fig:24:Barplot of OPAnnualDeductibleAmt vs target

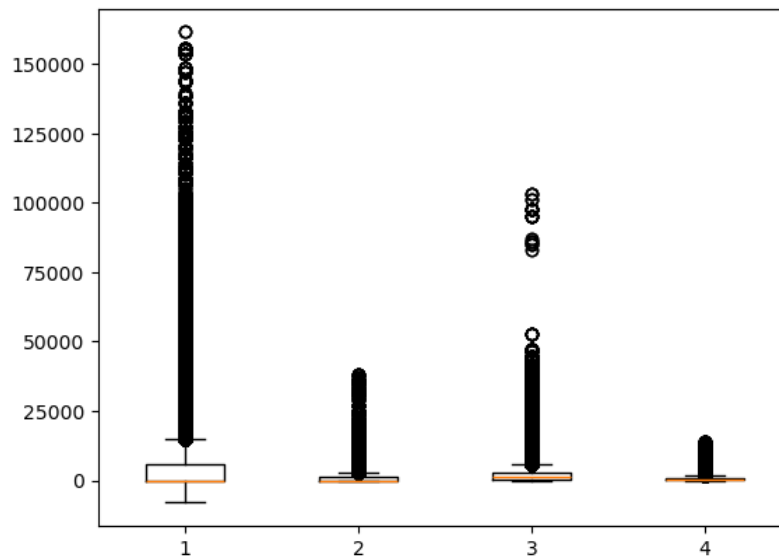


Fig:25 :boxplot of op and IP deductible and reimbursed amount

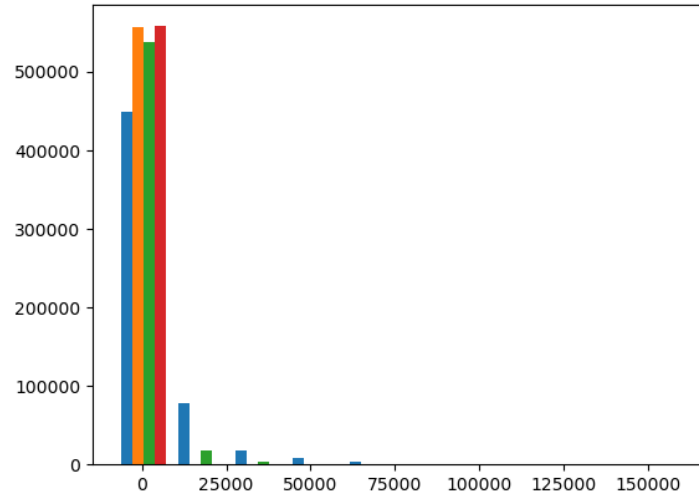


Fig:26:Histogram of op and ip reimbursed and deductible amount

Medical Information

26.ClmAdmitDiagnosisCode

Diagnosis code provided during admission. Used to understand the reason for admission. There are null values and have statistically significant association with the target column.

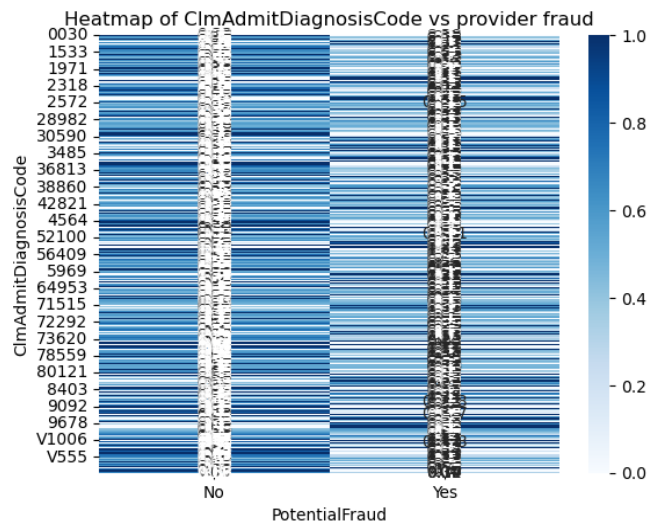


Fig27:heatmap of claimdiagnosiscode vs potential fraud

27. ClmDiagnosisCode_1 to ClmDiagnosisCode_10

Up to 10 diagnosis codes related to the claim. Helps in analyzing the patient's condition or treatment trends. There are null values present in these columns so we will deal with this in the preprocessing stage. All these columns have statistically significant association with the target, except **ClmDiagnosisCode_10**.

28. ClmProcedureCode_1 to ClmProcedureCode_6

Codes representing medical procedures performed. Useful for understanding the type and frequency of services. There are null values present in these columns; we will handle this in preprocessing.

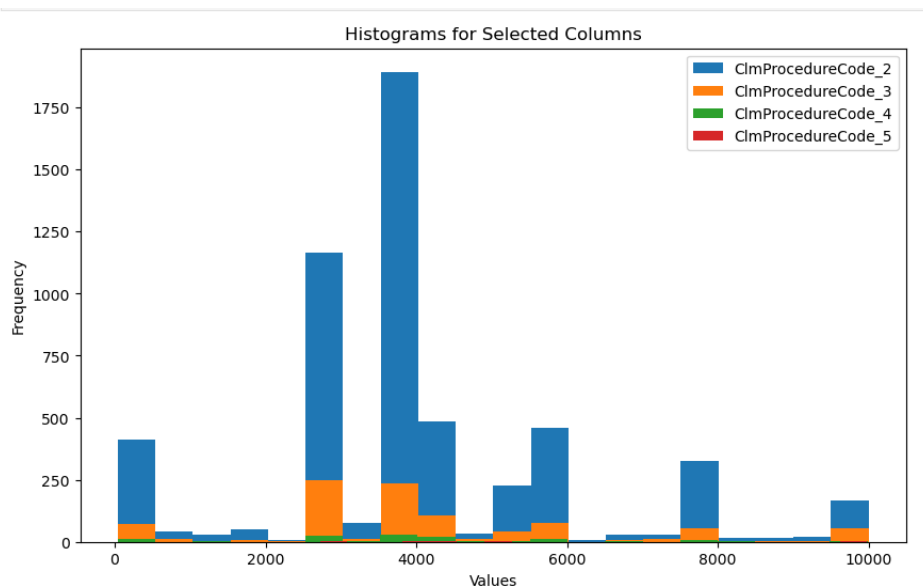


Fig:28:Histogram of claim procedure codes.

29. DiagnosisGroupCode

Broad grouping of diagnoses. Provides an overview of medical conditions. Broad grouping of diagnoses and these medical codes used to classify hospital cases and determine how much Medicare pays a hospital for each cases. This code Provides an overview of medical conditions. There is no statistically significant association between DiagnosisGroupCode and PotentialFraud.

```

Crosstab for DiagnosisGroupCode and PotentialFraud:
PotentialFraud          No      Yes
ClnAdmitDiagnosisCode
0030          0.500000  0.500000
0059          0.333333  0.666667
00845         0.543860  0.456140
0085          0.000000  1.000000
00863         1.000000  0.000000
...
V8279         0.000000  1.000000
V8281         0.625899  0.374101
V8289         0.684211  0.315789
V829          0.863636  0.136364
V854          1.000000  0.000000

[4098 rows x 2 columns]

```

Table 2 : crosstab of claimdiagnosisgroupcode vs target

Chronic Conditions

30.ChronicCond_Alzheimer

Indicates if the patient has Alzheimer's disease. Its data type is int64, discrete numerical. distribution is skewed.it has missing values and no outliers.it relates to potential fraud as follows:

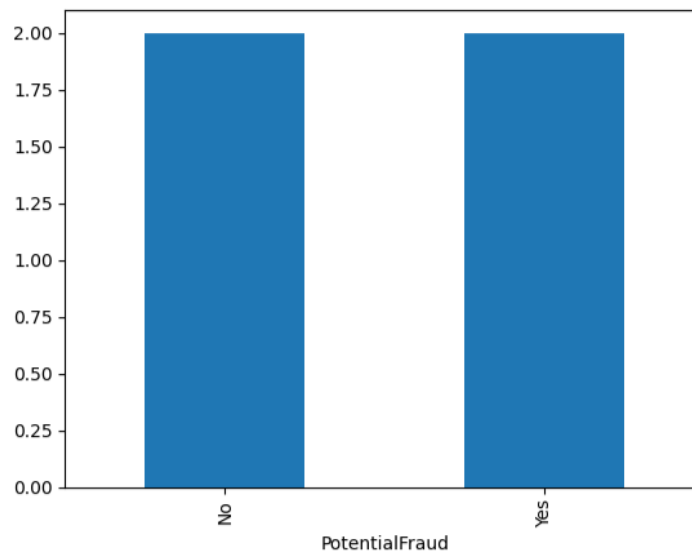


Fig:29.Barplot of ChronicCond_Alzheimer vs target

31. **ChronicCond_Heartfailure**

Indicates if the patient has history of heart failure (pre- existing Conditions). This is a discrete numerical column with only two unique values 1 and 2 which indicates yes and no, and there are no null values present in this.

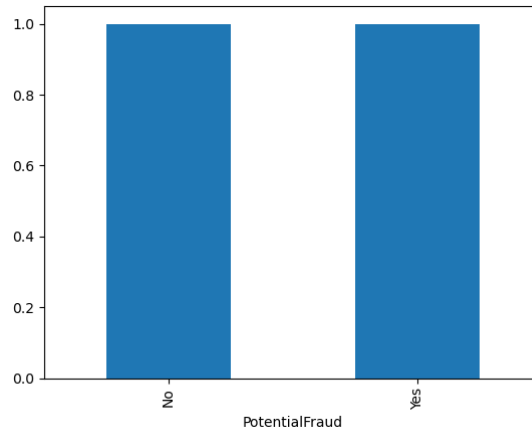


Fig 30:-Barplot ChronicCond_Heartfailure vs potentialfraud

32. **ChronicCond_KidneyDisease** Indicates if the patient has history of chronic kidney disease (pre- existing Conditions). This is a discrete numerical column with only two unique values 1 and 2 which indicates yes and no, and there are no null values present in this.

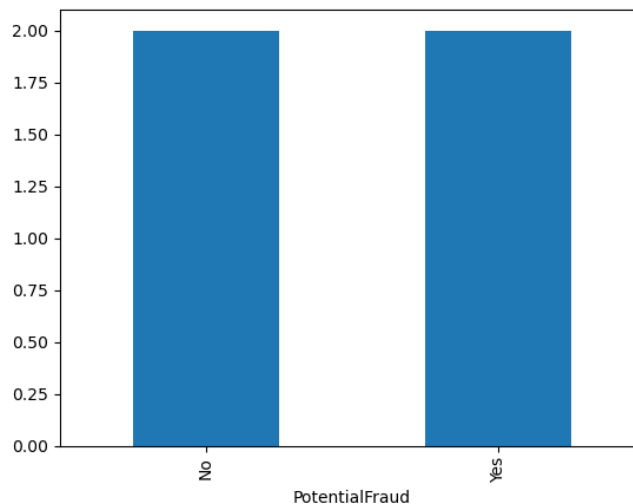


Fig:31.Barplot of **ChronicCond_KidneyDisease** vs Target

33. **ChronicCond_Cancer**. Indicates if the patient has history of cancer (pre- existing Conditions). This is a discrete numerical column with only two unique

values 1 and 2 which indicates yes and no, and there are no null values present in this.

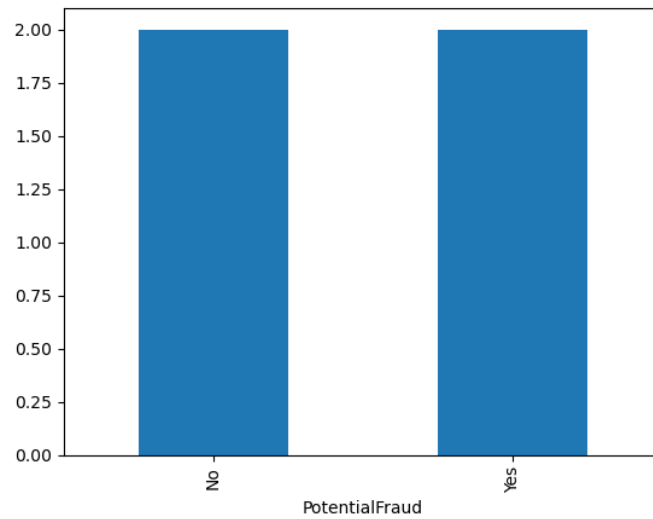


Fig:32:Barplot of **ChronicCond_Cancer** vs Target

34.ChronicCond_ObstrPulmonary. Indicates if the patient has history of obstructive pulmonary disease (pre- existing Conditions). This is a discrete numerical column with only two unique values 1 and 2 which indicates yes and no, and there are no null values present in this.

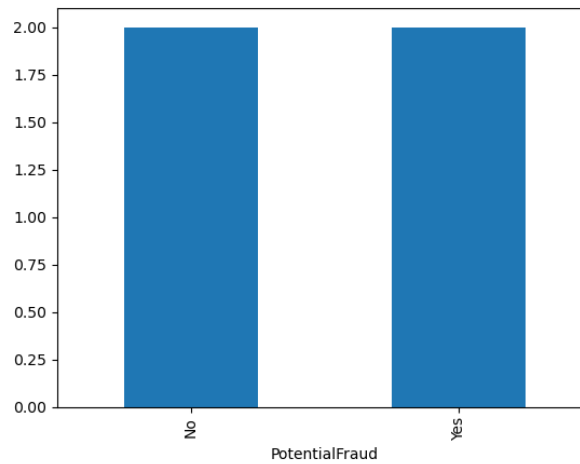


Fig:33.Barplot of **ChronicCond_ObstrPulmonary** vs Target

35.ChronicCond_Depression

Indicates if the patient has history of depression (pre- existing Conditions).

This is a discrete numerical column with only two unique values 1 and 2 which indicates yes and no, and there are no null values present in this.

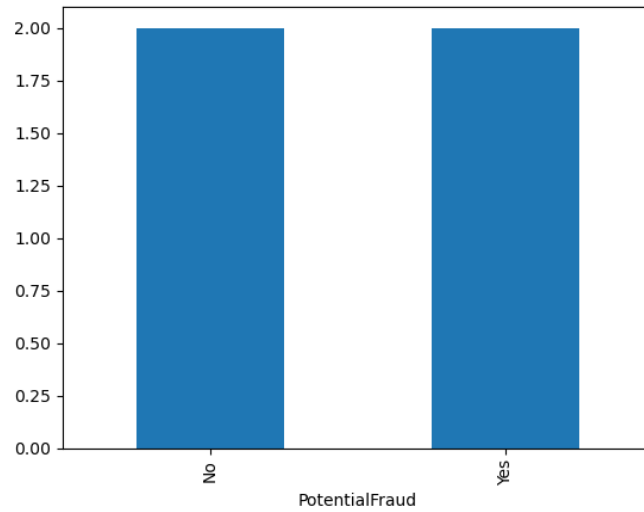


Fig:34.Barplot of ChronicCond_Depression vs target

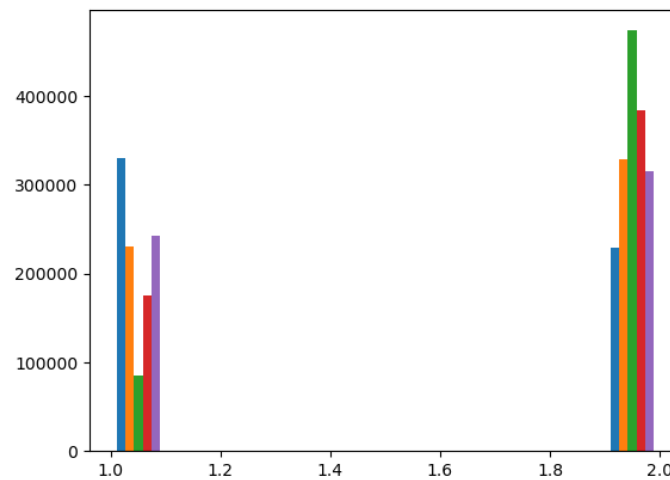


Fig35:Histogram chronicconditions

36.ChronicCond_Diabetes

Indicates if the patient has diabetes. This is a discrete numerical column with only two unique values 1 and 2 which indicates yes and no, and there are no null values present in this.

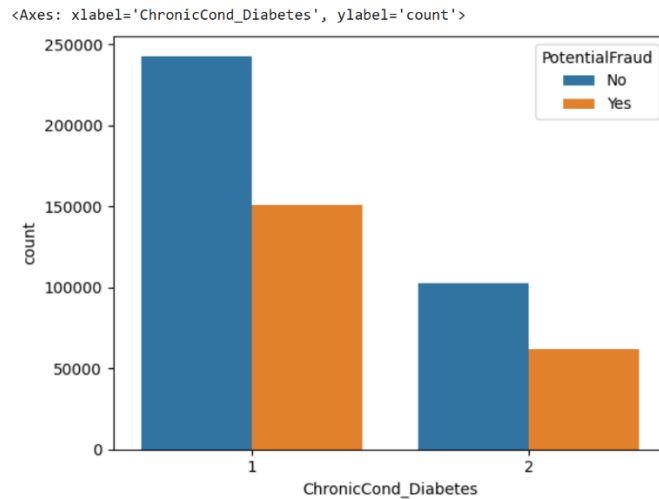


Fig.36: countplot of chronicdiabetes vs potentialfraud

37. ChronicCond_IschemicHeart

Indicates if the patient has ischemic heart disease. Discrete numerical column with no null values .2 unique values 1 and 2.

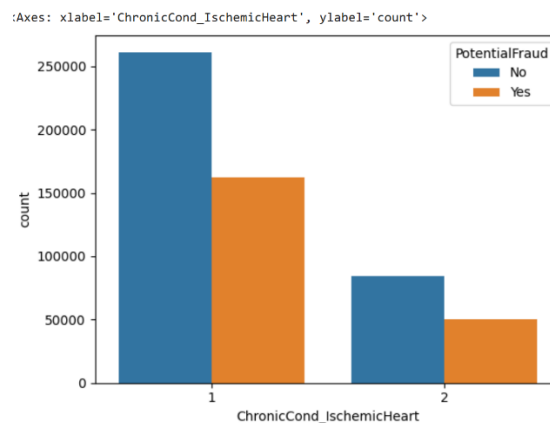


Fig:37: count plot of chroniccond_ischemicHeart vs potential fraud.

38. ChronicCond_Osteoporosis

Indicates if the patient has osteoporosis. Discrete numerical column with no null values, two unique values 1 and 2.

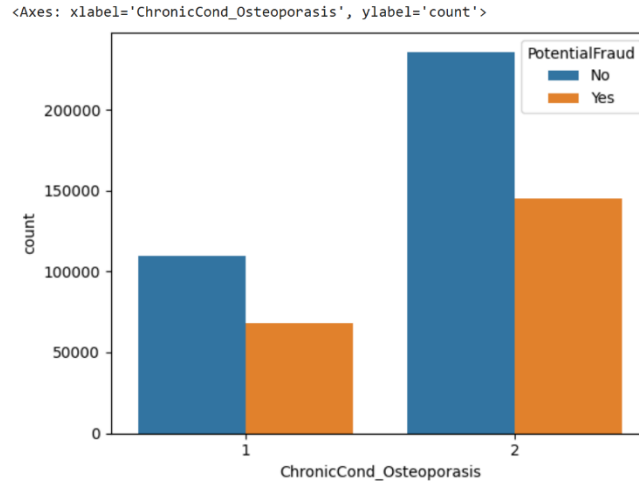


Fig: 38.countplot of chroniccond_osteoporosis vs potentialfraud.

39.ChronicCond_rheumatoidarthritis

Indicates if the patient has rheumatoid arthritis. Its data type is int64, discrete numerical. Distribution is skewed.it has missing values and has no outliers.it relates to potential fraud as follows:

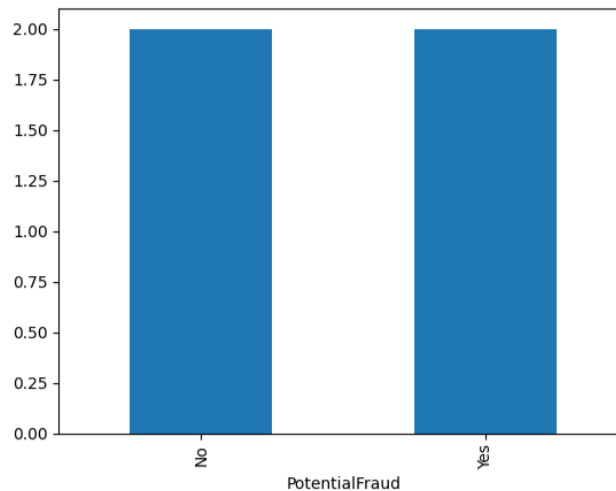


Fig:39.barplot of ChronicCond_rheumatoidarthritis vs Target

40.ChronicCond_stroke

Indicates if the patient has a history of stroke. Its data type is int64, discrete numerical. Distribution is skewed.it has missing values and has outliers.it relates to potential fraud as follows:

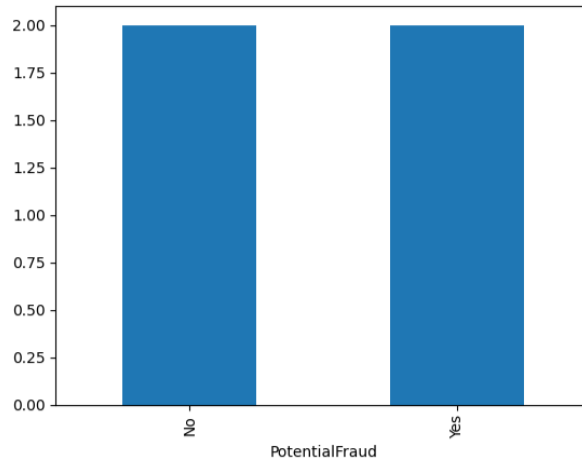


Fig 40:Barplot **ChronicCond_stroke** vs target

To identify relationships among the numerical features, we created a correlation matrix for all 29 numerical features and plotted a heatmap using that matrix. We observed a strong positive correlation among some of the numerical features.

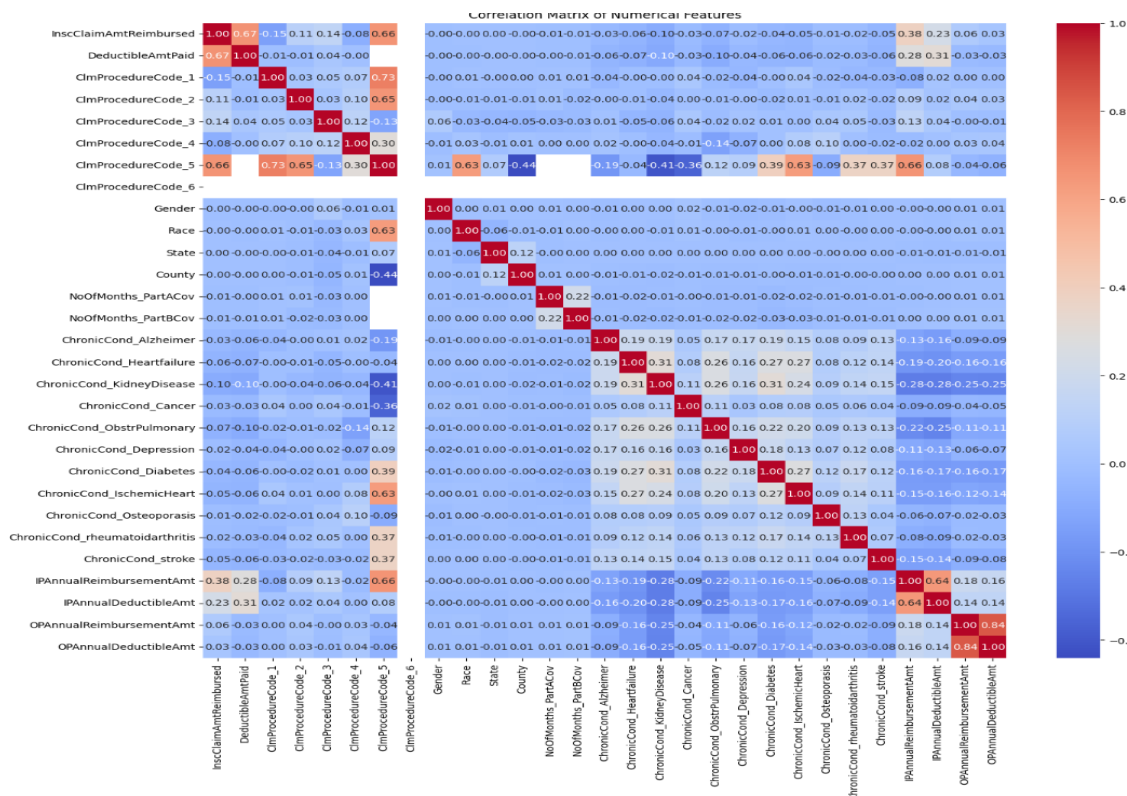


Fig 41:Heatmap of correlation matrix

After performing univariate analysis of each feature, we found that some columns contain a significant amount of null values (more than 99%). We will remove those columns based on their significance with the target. For other columns with negligible null values, we will fill them using non-null values by identifying patterns in the data. Additionally, some columns contain outliers; we will analyze the significance of these outliers on fraud prediction and address them in the preprocessing steps.

After performing bivariate analysis between all the features and the target column, we found that some columns have no statistical significance with the target and will not influence the model for predicting fraud. We will address this in the preprocessing steps.

- **Feature Engineering:**

Out of the 55 features in our healthcare dataset, we removed some columns with little to no influence on our target by examining the null value counts and the number of unique labels.

The column '**ClaimID**', which is unique for each claim, contains a total of 558,211 values, all of which are unique. Since it has no influence on predicting fraud, we dropped this column.

Some columns, such as **ClmProcedureCode_3 to 6** and **ClmDiagnosis-Code_10**, have 99% null values, so we dropped these columns. However, the '**DOD**' (date of death) column, despite having 99% null values, was not dropped because it is required for feature creation.

After feature selection, the number of features was reduced from 55 to 49. Null value in 'deductibleamt' is replaced by median based on its distribution. Null values in other columns were replaced with **0** for numerical columns and "**NONE**" for categorical columns, because some data are only present for outpatients and some for inpatients. In DOD column we cannot fill missing values with any other values, missing values in that column may be due to that patient is still living. Diagnosis codes and procedure codes also cannot be filled with any other codes, because some patients might not require all the procedures, and they might not have all the diagnoses, which is why those codes are missing. Therefore, we filled the missing values with zero.

Other null values were automatically removed during the feature engineering steps.

Handling of missing values follows the outlier detection by plotting box-plot. The box-plot of following columns indicates the outliers of the same.

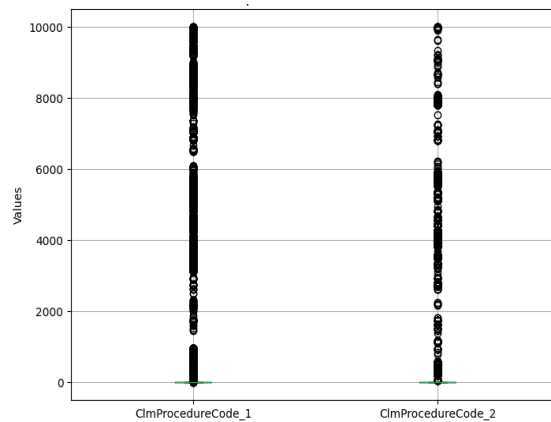


Fig 42 : Box-plot of columns Clmprocedurecodes 1 and 2

New Features Derived from Date Columns

There were 6 dates column in our dataset, they are '**ClaimStartDt**', '**ClaimEndDt**', '**AdmissionDt**', '**DischargeDt**', '**DOB**', '**DOD**' ,we have derived 4 features from these .

ClaimDuration : which is the difference between claim start date and end date ,which indicates the delay in settling a claim. By analyzing claim durations and comparing them to expected norms and patterns, we can identify unusual behaviors that may indicate fraud, such as overbilling, duplicate claims, or fabricated services

TotalLengthOfStay: We created this feature by taking the difference between Admission date and discharge date and only days are extracted from this. This feature will have non null values only for inpatients and will be zero for outpatients. After deriving this ,we dropped Admission date and discharge date columns .Missing values in length of stays are filled by zero in the assumption that those were outpatients .

AGE: The column DOB indicates date of birth. From this column, we derived a new column AGE, which shows the age of person. Then we removed the column DOB. There were no missing values in DOB feature so that in the new feature AGE also.

IsDead:The column DOD indicates date of death. From this columns ,we derived a new column IsDead, which shows whether a person is dead or not. Then we dropped the column DOD. There were missing values in DOD feature so that in the new feature IsDead also. We came into a conclusion that these missing values shows the person is not dead and hence we filled those missing values by 'NONE'.

Features Derived From Amount Related Columns

TotalClaimAmt: It is the sum of 'InscClaimAmtReimbursed' which is the amount paid by the insurance company and 'DeductibleAmtPaid' is the copayment that the beneficiary need to pay for that particular claim .adding these two will gives the total claim amount .

There were some missing values in the deductible amount column, which may be due to the fact that certain insurance policies cover the full claim amount without requiring any copayment from the beneficiary. Therefore, we filled those missing values with zero (0).

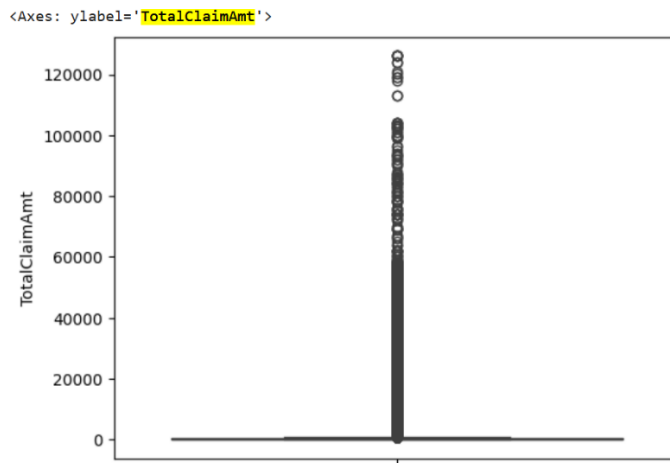


Fig 43: Box plot of TotalClaimAmt

From the above boxplot it is clear that so many outliers were present in Totalclaimamt column, These outliers represent claims with unusually high amounts compared to the majority. Most of the data points are concentrated near the bottom of the plot, suggesting that most claims have low to moderate

amounts. A small number of claims exceed 100,000 in the TotalClaimAmt. These extreme values could potentially indicate:

- Legitimate high-cost claims (e.g., for complex or severe medical procedures).
- Fraudulent claims if the values are unusually high and inconsistent with the services provided.

Claims with significantly higher amounts than average should be investigated. Such patterns might indicate:

- Billing for services not rendered.
- Upcoding (billing for more expensive procedures than performed).
- Duplicate claims or other fraudulent activities.

We are not removing any outliers because of their significance in fraud prediction.

OPtotalamt: It is the sum of 'OPAnnualReimbursementAmt' and 'OPAnnualDeductibleAmt' for the outpatient only. It represents the total outpatient claim amount.

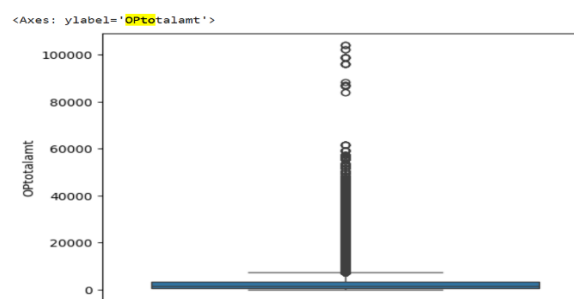


Fig 44:Boxplot of optotalamt

Most claims have relatively low outpatient amounts, as shown by the tight clustering near the bottom of the plot. Outliers have a significant impact on fraud detection, so we are not removing them, as in the case of the total claim amount.

TotalAnnualIPAmount: It is the sum of 'IPAnnualReimbursementAmt' and 'OPAnnualDeductibleAmt' for the inpatients. The majority of the data points fall within a relatively small range near the bottom of the plot, as indicated by the small height of the box. Outliers are present in this amount but due to their significance in fraud prediction we are not handling this in preprocessing.

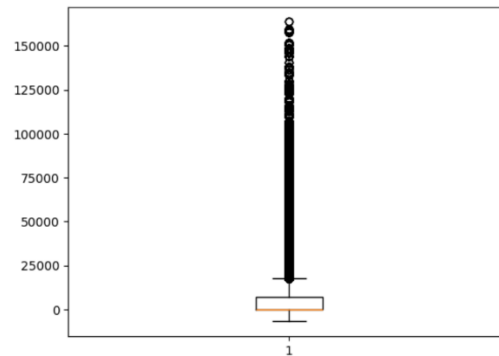


Fig 45: Boxplot of TotalAnnualIPAmount

Feature Derived from Physician related columns

There were three columns related to physicians, such as 'AttendingPhysician,' 'OperatingPhysician,' and 'OtherPhysician.' From these, we created six new features that will be helpful in detecting fraudulent claims.

PhysicianOverlap: Captures cases where the same physician is listed in multiple roles (e.g., as both attending and operating). This is a straightforward anomaly indicator.

PhysicianInfoMissing: Identifies claims with incomplete physician details, as missing information might suggest fraudulent intent.

AttendingPhysicianFrequency, OperatingPhysicianFrequency, OtherPhysicianFrequency: Physicians with unusually high claim activity might indicate suspicious behavior.

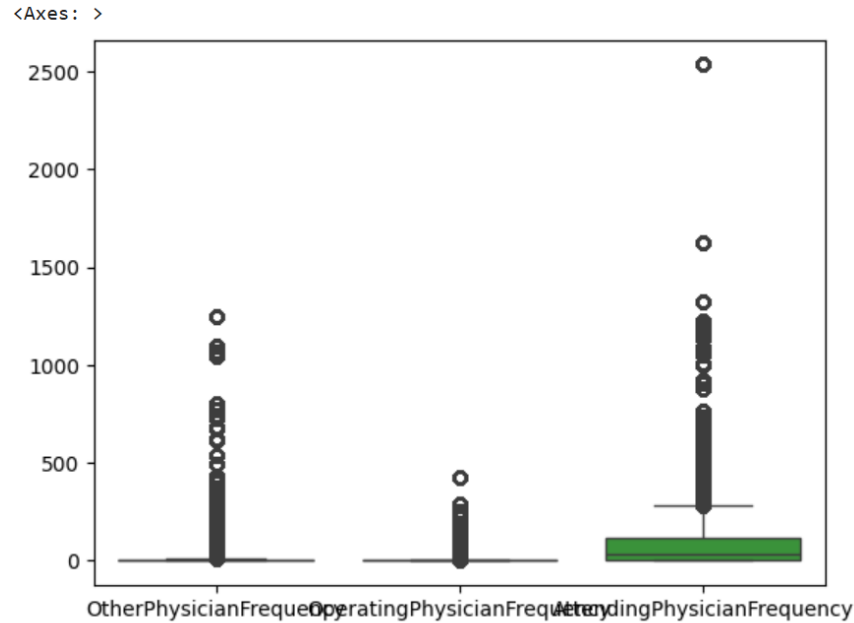


Fig 46:Box plot of physician appearing frequency.

From the above plot, ‘AttendingPhysicianFrequency’ has the most prominent spread (a larger box), suggesting a higher variability in its values. ‘OtherPhysicianFrequency’ and ‘OperatingPhysicianFrequency’ have narrower boxes, showing less variability. There are significant outliers in all categories, with ‘AttendingPhysicianFrequency’ having the highest outliers extending above 2500. We are not removing these outliers due to its significance in detecting fraud.

PhysicianRoleCount

This feature counts the number of unique physicians involved in a single claim, based on the three columns (AttendingPhysician, OperatingPhysician, OtherPhysician). The idea is that a claim involving more unique physicians might be a legitimate medical claim, whereas a claim with too few unique physicians (e.g., only one or two physicians across all roles) could be suspicious.

IS_inpatient: One more feature is created to distinguish between inpatient and outpatient.

ANOVA TEST: Done to check the significance of new derived and existing numerical columns with the target column and found that 6 columns(Gender, NoOfMonths_PartBCov,ChronicCond_Depression,ChronicCond_Osteoporosis,OPtotalamt, is dead) were not significant. Hence these columns were removed and finally we got 42 columns.

Below are the 6 columns we removed because P value >0.05 after cross checking with the correlation matrix.

OPtotalamt	2.983576e+00	8.411379e-02
NoOfMonths_PartBCov	2.142225e+00	1.432945e-01
is_dead	9.795747e-01	3.223042e-01
ChronicCond_Osteoporosis	7.779602e-01	3.777661e-01
ChronicCond_Depression	1.219878e-01	7.268886e-01
Gender	1.178654e-01	7.313614e-01

Table 3- ANOVA Test Result-Columns with P value > 0.05

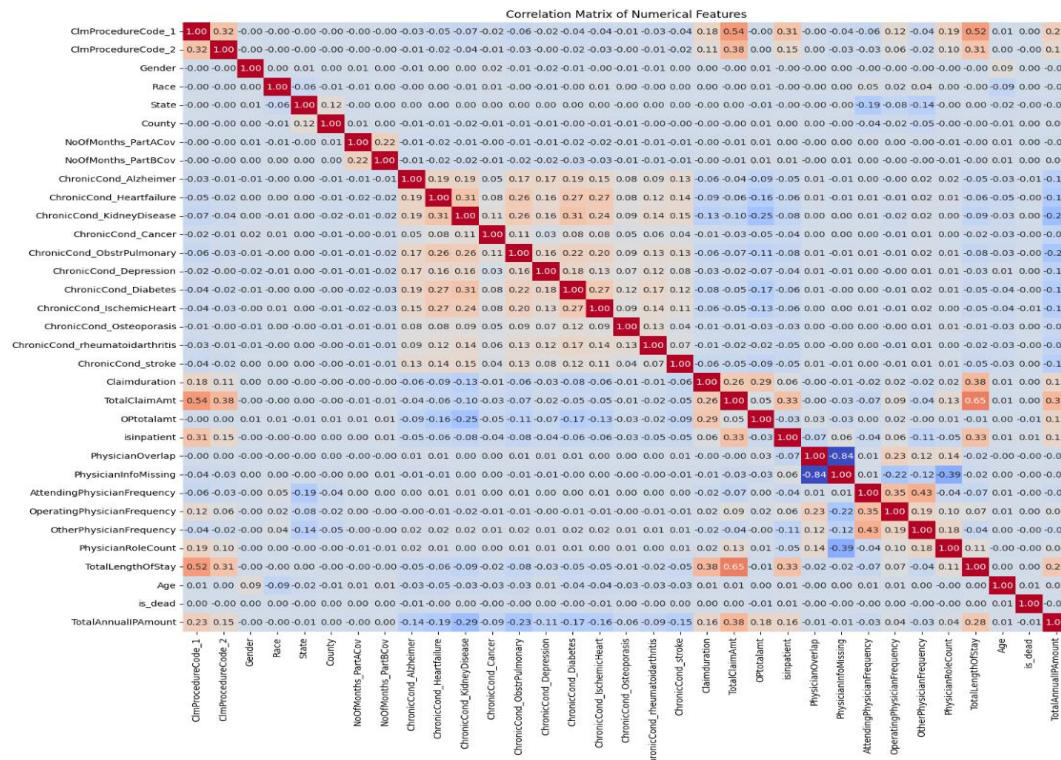


Fig:47. Correlation matrix after feature engineering

ENCODING

We have done different encoding for different categorical features in our dataset.

Target column is Label encoded: ‘PotentialFraud’ is our Target column. The ‘PotentialFraud’ column was label-encoded, where "YES" was mapped to 1 and "NO" was mapped to 0.

BeneID column is target encoded : Target encoding is particularly useful for categorical features with many unique categories. In our dataset, the ‘BeneID’ column contains 138,556 unique labels. We applied target encoding with respect to our target column, ‘PotentialFraud’ The ‘PotentialFraud’ column was label-encoded, where "YES" was mapped to 1 and "NO" was mapped to 0. For target encoding, the mean of the target (PotentialFraud) was calculated for each unique ‘BeneID’, and each ‘BeneID’ was replaced by its corresponding mean. As a result, the target-encoded column has values in the range of 0 to 1.

To address the issue of extreme values (e.g., 0) for less frequent IDs, we applied a smoothing technique during target encoding. This ensured more stable estimates for IDs with fewer occurrences.

Provider column is one hot encoded: The Provider column was one-hot encoded. Initially, the column contained 5,410 unique IDs. We first analyzed the value counts for each provider ID and replaced those appearing fewer than 3,000 times in the dataset with a label, less frequent. This reduced the number of unique values to 7: the 6 most frequent provider IDs and the less frequent label. After applying one-hot encoding, we created 7 new features, each taking values of 0 or 1.

Ordinal Encoding: We have applied ordinal encoding for ‘DiagnosisGroup-Code’ to Maintain severity/order relationships in diagnosis group code that may be critical for fraud detection.

We choose this encoding because If the diagnosis codes represent an ordered relationship (e.g., severity: mild < moderate < severe), ordinal encoding ensures

this order is maintained numerically. This can help models understand relationships.

Frequency Encoding: we used frequency encoding for these columns 'ClmAdmitDiagnosisCode', 'ClmDiagnosisCode 1 to 9' because these columns often have many unique values (e.g., different diagnosis codes). Frequency encoding reduces dimensionality without creating excessive new columns and can be correlated with the likelihood of fraud.

We also replaced Y values as 1 in column 'RenalDiseaseIndicator' and converted it into 'int' datatype, now this column contains 1 and 0, before it was y and 0. Where y indicates the presence of renal disease.

PICKLING: all the encoder used in training the models are pickled for save and reuse the encoding mappings or processed data across different stages of a project to avoid errors due to mismatched mappings between training and test datasets.

SMOTE(OVERSAMPLING) TO REMOVE IMBALANCE: used to address class imbalance in datasets. This **Improves Model Performance** by balancing the dataset, enabling models to learn from both classes effectively and generates synthetic data instead of simply duplicating minority class samples. After SMOTE our dataset consists of **690830 observations** and **48 features**.



	proportion
PotentialFraud	
0	0.5
1	0.5

dtype: float64

Table 4: Target label proportion after oversampling

SCALING

After encoding, we moved to the scaling of features. For scaling, we applied standard scaler except for the encoded features and the Target column. Numerical features typically need scaling because they may vary significantly in magnitude and units. Some models are sensitive to the scale of numerical features,

so standardization ensures all features contribute equally. After scaling ,all numerical values came into a range between -1 and 1 ,mean 0 and standard deviation 1. Scaling is applied to both normal and over sampled data after encoding.

	0	1	2	3	4	5
count	5.582110e+05	5.582110e+05	5.582110e+05	5.582110e+05	5.582110e+05	5.582110e+05
mean	2.615795e-17	-1.135420e-17	9.113912e-17	5.784025e-17	-2.382855e-17	-5.673028e-16
std	1.000001e+00	1.000001e+00	1.000001e+00	1.000001e+00	1.000001e+00	1.000001e+00
min	-1.845628e-01	-8.924226e-02	-3.554479e-01	-1.609119e+00	-1.427475e+00	-1.341051e+01
25%	-1.845628e-01	-8.924226e-02	-3.554479e-01	-9.509108e-01	-8.618967e-01	7.702261e-02
50%	-1.845628e-01	-8.924226e-02	-3.554479e-01	-9.524063e-02	-1.077924e-01	7.702261e-02
75%	-1.845628e-01	-8.924226e-02	-3.554479e-01	8.262503e-01	7.217224e-01	7.702261e-02
max	7.310720e+00	2.200588e+01	5.219963e+00	1.879383e+00	2.339276e+00	7.702261e-02

8 rows x 27 columns

Table 5: Description of oversampled data after scaling

MODELING: We have applied below mentioned models to our original data and over sampled data as well.

Original Data:

1. Logistic Regression (LR)

- Accuracy: 0.74
- Precision: 0.73
- Recall: 0.50
- F1 Score: 0.59

2. K-Nearest Neighbors (KNN)

- Accuracy: 0.74
- Precision: 0.64
- Recall: 0.49
- F1 Score: 0.55

3. Naive Bayes (NB)

- Accuracy: 0.69
- Precision: 0.83
- Recall: 0.24
- F1 Score: 0.38

4. Decision Tree (DT)

- Accuracy: 0.74
- Precision: 0.73

- Recall: 0.50
 - F1 Score: 0.59
5. **Random Forest (RF)**
- Accuracy: 0.74
 - Precision: 0.73
 - Recall: 0.50
 - F1 Score: 0.59
6. **AdaBoost:**

	precision	recall	f1-score	support
0	0.88	0.83	0.86	69258
1	0.75	0.81	0.78	42385
accuracy			0.83	111643
macro avg	0.81	0.82	0.82	111643
weighted avg	0.83	0.83	0.83	111643

Table 6:performance metrics of ADA boost classifier (data without oversampling)

confusion matrix $\begin{bmatrix} 57746 & 11512 \\ 7920 & 34465 \end{bmatrix}$ confusion matrix adaboost normal data

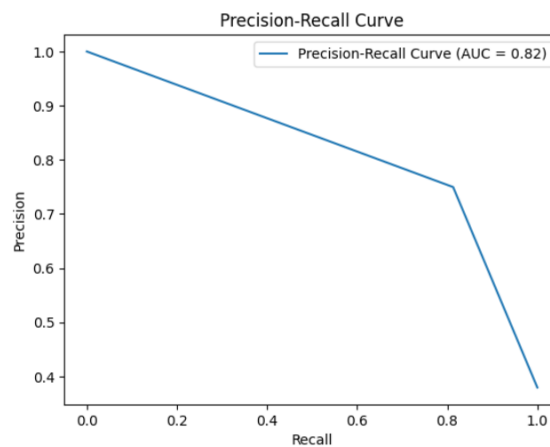


Fig 48:Precision-Recall curve of ADA boost classifier for normal data

7.Light GBM

	precision	recall	f1-score	support
0	0.87	0.94	0.91	69258
1	0.89	0.78	0.83	42385
accuracy			0.88	111643
macro avg	0.88	0.86	0.87	111643
weighted avg	0.88	0.88	0.88	111643

confusion matrix

```
[[65244 4014]
 [ 9394 32991]]
```

Table 7:performance metrics of Light GBM classifier (data without oversampling)

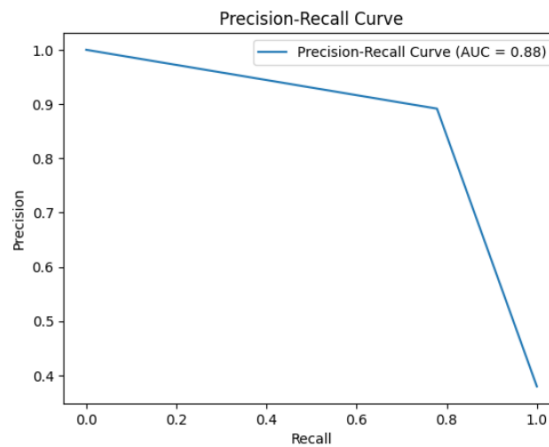


Fig 49:Precision-Recall curve of Light GBM classifier for normal data

Over sampled data:

1. Logistic Regression (LR)

- Accuracy: 0.78
- Precision: 0.81
- Recall: 0.73
- F1 Score: 0.77

2. K-Nearest Neighbors (KNN)

- Accuracy: 0.71
- Precision: 0.73
- Recall: 0.67
- F1 Score: 0.70

3. Naive Bayes (NB)

- Accuracy: 0.61
- Precision: 0.89
- Recall: 0.25
- F1 Score: 0.39

4. Decision Tree (DT)

- Accuracy: 0.85
- Precision: 0.85
- Recall: 0.86
- F1 Score: 0.85

5. Random Forest (RF)

- Accuracy: 0.88
- Precision: 0.90
- Recall: 0.87
- F1 Score: 0.88

6. AdaBoost:

	precision	recall	f1-score	support
0	0.86	0.78	0.82	68991
1	0.80	0.88	0.84	69175
accuracy			0.83	138166
macro avg	0.83	0.83	0.83	138166
weighted avg	0.83	0.83	0.83	138166

Table 8: performance metrics of ADA boost classifier (data with oversampling)

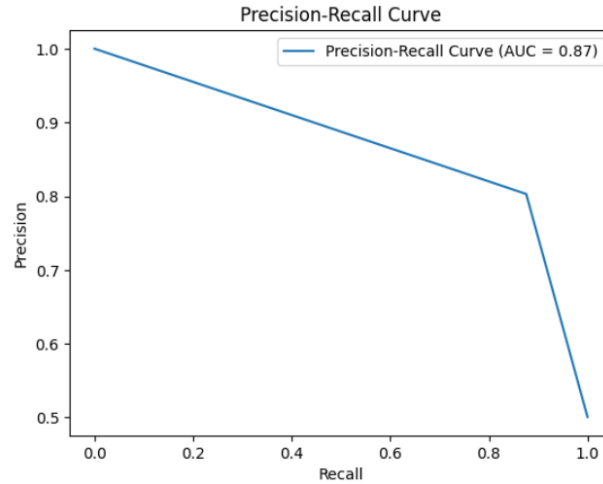


Fig 50: Precision-Recall curve of ADA boostclassifier for oversampled data

confusion matrix $\begin{bmatrix} 54128 & 14863 \\ 8603 & 60572 \end{bmatrix}$ confusion matrix for ada boost oversampled

7.Light GBM for oversampled data

	precision	recall	f1-score	support
0	0.87	0.93	0.90	68991
1	0.92	0.86	0.89	69175
accuracy			0.89	138166
macro avg	0.89	0.89	0.89	138166
weighted avg	0.89	0.89	0.89	138166
confusion matrix	$\begin{bmatrix} 63838 & 5153 \\ 9628 & 59547 \end{bmatrix}$			

Table 9: performance metrics of Light GBM classifier (data with oversampling)

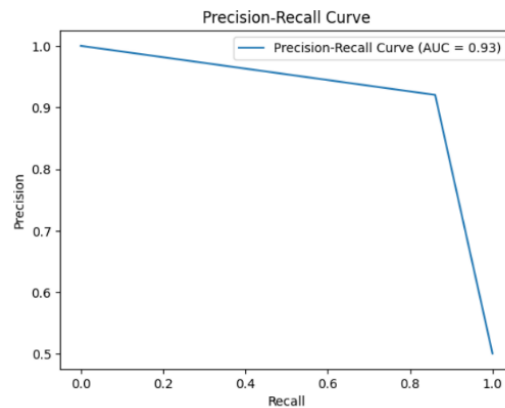


Fig 51: Precision-Recall curve of Light GBM classifier for oversampled data

XGBoost: Finally, we have identified XGBoost as the best-performing model for our dataset. Hence, we decided to do hyperparameter tuning for original data and over sampled data as well. Details given below in the model evaluation.

Hyperparameter Tuning:

1. Parameter Grid for XGBoost

The `param_grid_xgb` specifies the range of hyperparameters to explore during the tuning process. Each parameter affects the model's performance in different ways:

- **n_estimators:**
 - Specifies the number of trees (or boosting rounds) in the model.
 - Higher values can improve accuracy but may lead to overfitting or increased training time.
 - Grid: [100, 200, 300].
- **learning_rate:**
 - Determines the step size for updating model weights after each boosting round.
 - Smaller values make the model learn slowly, requiring more boosting rounds but potentially leading to better generalization.
 - Grid: [0.01, 0.1, 0.2].
- **max_depth:**
 - Limits the maximum depth of each tree.
 - Prevents overfitting by controlling the complexity of the model.
 - Grid: [3, 5, 7].
- **subsample:**
 - Fraction of samples used to grow each tree.
 - Reducing this value helps prevent overfitting by introducing randomness.

- Grid: [0.8, 0.9, 1.0].
- **colsample_bytree:**
- Fraction of features randomly selected for each tree.
- Helps in reducing overfitting and makes the model robust.
- Grid: [0.8, 0.9, 1.0].

2. RandomizedSearchCV Parameters

The RandomizedSearchCV performs a random search over the specified parameter grid.

- **estimator:**
- The machine learning model to be optimized. Here, it is XGB Classifier.
- **param_distributions:**
- The parameter grid defines the range of values to be tested.
- **n_iter:**
- Number of random parameter combinations to evaluate.
- Set to 5 for faster computation. Increasing it improves the chances of finding better parameters.
- **scoring:**
- Metric to evaluate each parameter combination.
- Using f1 (harmonic mean of precision and recall) since it's suitable for imbalanced data.
- **cv:**
- Number of folds for cross-validation.
- Set to 3 to balance computational cost and evaluation accuracy.
- **verbose:**
- Controls the verbosity of the output.
- Set to 2 for detailed updates during the process.
- **n_jobs:**
- Number of CPU cores to use for parallel processing.
- Set to -1 to utilize all available cores.
- **random_state:**
- Ensures reproducibility by fixing the random seed.

3. Results

After fitting the RandomizedSearchCV on the training data:

- **random_search_xgb.best_params_:**
 - Printed the best combination of hyperparameters found.
- **random_search_xgb.best_score_:**
 - Displayed the best F1-score achieved during cross-validation.

After all these processes we got the best Tuned Parameters:
subsample=0.9, n_estimators=200, max_depth=7, learning_rate=0.2, colsample_bytree=1.0

EVALUATION OF MODEL:

1. XGBoost on Original Data

Without Hyperparameter Tuning

- Performance Metrics:
 - **Precision (class 1):** 91%
 - **Recall (class 1):** 80%
 - **F1 Score (class 1):** 85%
 - **Accuracy:** 89%
- **Confusion Matrix:**
 - True Negatives (Class 0): 65,733
 - False Positives (Class 0): 3,525
 - False Negatives (Class 1): 8,662
 - True Positives (Class 1): 33,723
- **Precision-Recall AUC:** Calculated and plotted

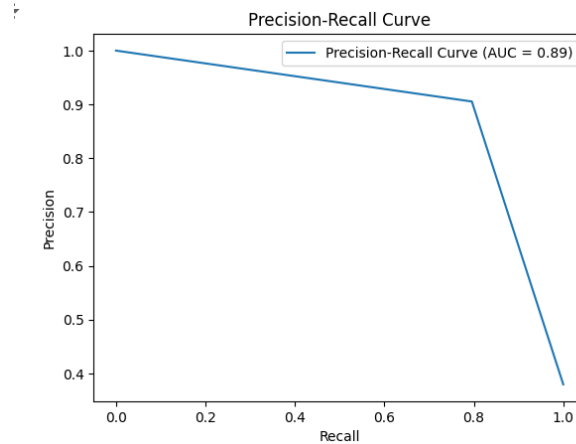


Fig:52. Precision -Recall curve XGboost for Original Data without Hyperparameter Tuning

With Hyperparameter Tuning

- **Precision-Recall Metrics:**

- Precision (class 1): 92%
- Recall (class 1): 81%
- F1 Score (class 1): 86%
- Accuracy: 90%

- **Confusion Matrix:**

- True Negatives (Class 0): 66,290
- False Positives (Class 0): 2,968
- False Negatives (Class 1): 8,133
- True Positives (Class 1): 34,252

Improved recall and precision slightly, reducing false negatives.

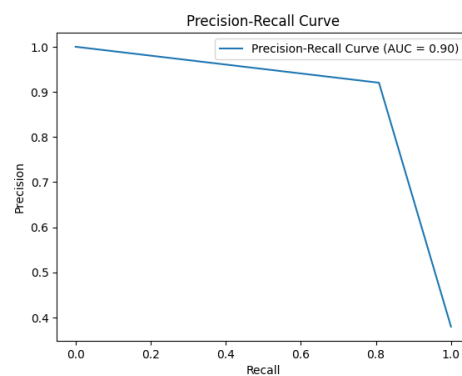


Fig:53. Precision -Recall curve XGboost for Data without Hyperparameter Tuning

2. XGBoost on Oversampled Data

Without Hyperparameter Tuning

- **Precision-Recall Metrics:**
 - **Precision (class 1):** 93%
 - **Recall (class 1):** 87%
 - **F1 Score (class 1):** 90%
 - **Accuracy:** 90%
- **Confusion Matrix:**
 - True Negatives (Class 0): 64,682
 - False Positives (Class 0): 4,309
 - False Negatives (Class 1): 8,919
 - True Positives (Class 1): 60,256
- **Precision-Recall AUC:** Calculated and plotted.

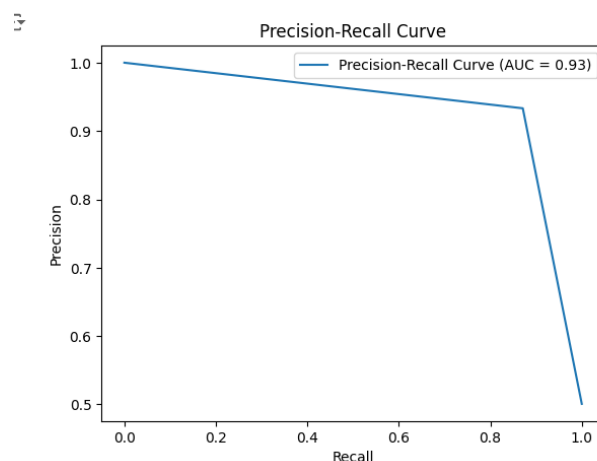


Fig:54. Precision-Recall curve for Over Sampled Data without Hyperparameter Tuning

With Hyperparameter Tuning

- **Precision-Recall Metrics:**
 - **Precision (class 1):** 94%
 - **Recall (class 1):** 88%
 - **F1 Score (class 1):** 91%
 - **Accuracy:** 91%
- **Confusion Matrix:**
 - True Negatives (Class 0): 65,355

- False Positives (Class 0): 3,636
- False Negatives (Class 1): 8,259
- True Positives (Class 1): 60,916

Further improved precision and recall, reducing false negatives.

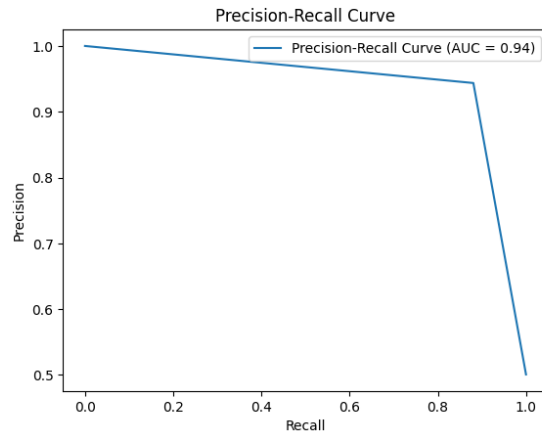


Fig:55. Precision-Recall curve for Over Sampled Data without Hyperparameter Tuning

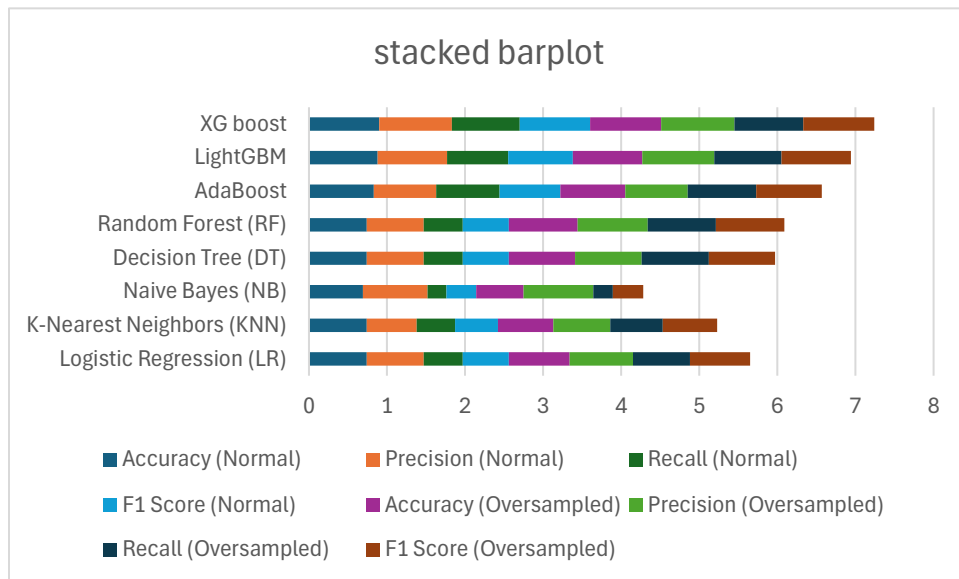


Fig:56. Performance metrics stacked barplot for various models

RESULTS

Model Performance: We identified XGBoost as the best-performing model for our dataset. We applied the model to both the original data and the oversampled data, testing it with and without hyperparameter tuning. We have used Random Search as the method for hyperparameter tuning. Ultimately, the best results were achieved using the oversampled data with hyperparameter tuning, yielding the highest precision of 94% balancing recall effectively with a slight boost in accuracy.

Key Findings: Feature Importance-The most important features derived from the test data includes isinpatient, Provider_less frequent, BenID and Diagnosis Group Code. These significantly impact the model's ability to detect fraud in test data. Chronic conditions like ChronicCond_Heartfailure show moderate importance. Since the analysis is based on test data, it confirms the model's generalization ability and highlights reliable predictors for unseen data.

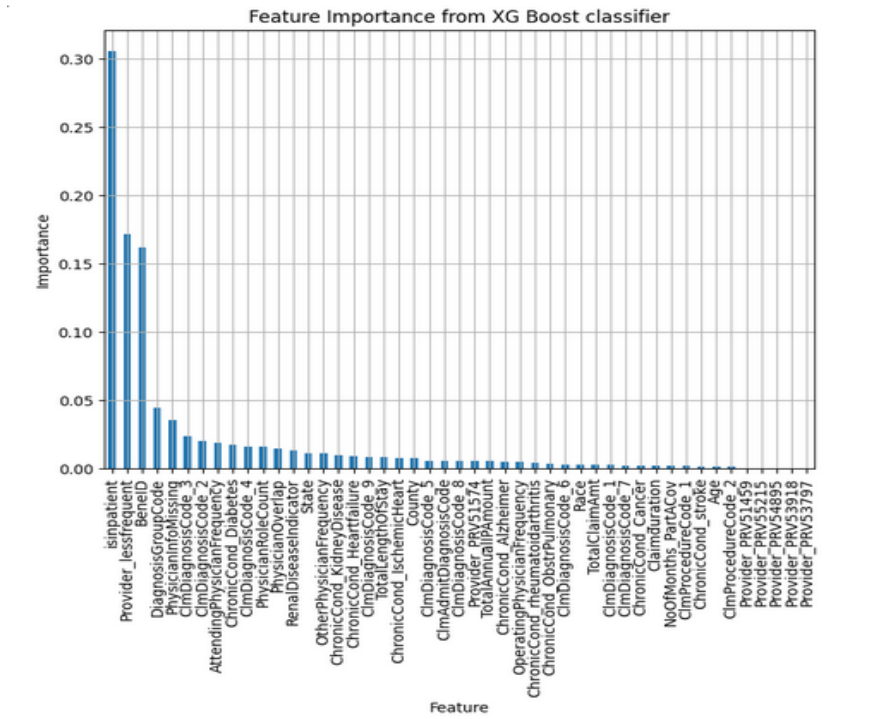


Fig:57. Feature Importance derived from Test data

Visualization: We developed web applications by using VS code and hosted using Python Anywhere. Then we uploaded test data as CSV file to the web application

and displayed the result as similar number of fraudulent (Yes) and non-fraudulent (No) cases, indicating effective handling of class imbalance (e.g., via over-sampling). The distinct peaks suggest strong class differentiation with minimal overlap. The sharp distribution implies confident predictions for both classes. The frequency distribution suggests a strong ability to predict both fraudulent and non-fraudulent cases. Validated results using metrics like precision, recall, F1-score, and AUC to ensure effectiveness.

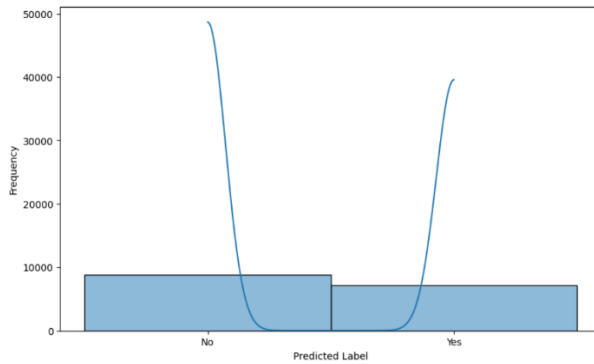


Fig:58. Histogram of Predicted labels in web page.

Website Overview

We have developed a web application using the Python Flask framework that allows users to upload a CSV file for analyzing fraud cases. The application processes the data, displays the results in an HTML table, and provides an option to download the analyzed data as a CSV file. The development was carried out using Visual Studio Code, and hosted the web app using pythonanywhere.



Fig:59. Website Homepage

Discussion

- **Limitations:** While acquiring data, we obtained separate datasets containing medical details. Merging these datasets resulted in a large dataset, which required significant memory and processing power. Training models became computationally expensive and time-consuming. Iterative tasks, such as hyperparameter tuning, were very slow and frequently caused Google Colab to crash.

The data we received was highly imbalanced, which posed a challenge, particularly in fraud detection, as it led to biased models.

While running our web application, we encountered **latency issues**. We had to wait for some time to get the result page after the data processing and analysis steps.

- **Future Work:** In our experiments, we observed better model performance with algorithms such as LightGBM, AdaBoost, Random Forest Classifier, and XGBoost. These models demonstrated strong predictive capabilities and effectively handled the challenges associated with our imbalanced dataset. To further enhance the robustness and accuracy of our predictions, we are planning to develop an ensemble model. The ensemble approach will combine the strengths of these individual models to leverage their complementary advantages.

Stacking: Combining the outputs of these models using a meta-model to improve overall performance.

Voting: Implementing either hard or soft voting to aggregate predictions from multiple models.

Blending: Using a weighted combination of predictions to prioritize the models that performed best on validation data.

This ensemble model will be evaluated rigorously to ensure it addresses our primary goal of minimizing false negatives, which is critical in fraud detection. By integrating this approach into our workflow, we anticipate not only improved accuracy but also a more robust system capable of handling real-world data complexities.

- **Implications: Enhanced Fraud Detection Accuracy:** Using the XGBoost model has shown strong performance, especially with complex and imbalanced datasets. It can identify detailed patterns in the data, making it better at detecting fraud and accurately identifying fraudulent claims.

Minimization of False Negatives: XGBoost reduces errors, especially false negatives, ensuring most fraudulent cases are detected. This is crucial in healthcare fraud detection to avoid financial and legal problems for the organization.

XGBoost's speed and scalability allow the system to handle large datasets typical of healthcare claims processing. Its ability to parallelize computations ensures that fraud detection can be performed efficiently, even in high-volume environments.

By accurately detecting fraudulent claims, our Healthcare Provider Fraud Detection app reduces financial losses for the organization, leading to significant cost savings. It also minimizes the need for extensive manual reviews, optimizing resource allocation.

Conclusions

Summary of findings: The XGBoost model has proven to be an effective solution for fraud detection in the healthcare sector, offering a balance between accuracy and operational efficiency. Future improvements may involve exploring ensemble techniques to further enhance model performance.

Giving a csv file as input to the web application eliminated the need for manually entering each parameter of a healthcare data set. Despite the advantages, the web application experiences some latency, particularly in the data processing and analysis stages. This delay can affect the overall user experience, especially when dealing with large datasets.

Impact: The potential impact of this project extends far beyond just fraud detection. It has the power to reshape how healthcare providers, insurers, and regulatory bodies approach fraud prevention, leading to significant improvements in efficiency, financial management, and overall trust in the system.

Recommendations: combining XGBoost with other models in an ensemble approach could further improve accuracy. To address the latency issue in the web application, optimizing processing speed using techniques like parallel processing and load balancing would be beneficial.

References

- Kaggle, Health care provider fraud dataset
- [Get Started with XGBoost — xgboost 2.1.3 documentation](#)
- <https://machinelearningmastery.com/>